



Guaranteeing the $\tilde{O}(\frac{AGM}{OUT})$ Runtime for Uniform Sampling and Size Estimation over Joins

Kyoungmin Kim

kmkim@dblab.postech.ac.kr

Pohang University of Science and Technology (POSTECH)
Pohang, Republic of Korea

George Fletcher

g.h.l.fletcher@tue.nl

Eindhoven University of Technology (TU/e)
Eindhoven, Netherlands

Jaehyun Ha

jha@dblab.postech.ac.kr

Pohang University of Science and Technology (POSTECH)
Pohang, Republic of Korea

Wook-Shin Han*

wshan@dblab.postech.ac.kr

Pohang University of Science and Technology (POSTECH)
Pohang, Republic of Korea

ABSTRACT

We propose a new method for estimating the number of answers OUT of a small join query Q in a large database D , and for uniform sampling over joins. Our method is the first to satisfy all the following statements.

- Support arbitrary Q , which can be either acyclic or cyclic, and contain binary and non-binary relations.
- Guarantee an arbitrary small error with a high probability always in $\tilde{O}(\frac{AGM}{OUT})$ time, where AGM is the AGM bound (an upper bound of OUT), and $\tilde{O}$ hides the polylogarithmic factor of input size.

We also explain previous join size estimators in a unified framework. All methods including ours rely on certain indexes on relations in D , which take linear time to build offline. Additionally, we extend our method using generalized hypertree decompositions (GHDs) to achieve a lower complexity than $\tilde{O}(\frac{AGM}{OUT})$ when OUT is small, and present optimization techniques for improving estimation efficiency and accuracy.

CCS CONCEPTS

• Information systems → Database query processing; • Mathematics of computing → Probability and statistics.

KEYWORDS

Join size estimation, Uniform sampling, Worst-case optimal

ACM Reference Format:

Kyoungmin Kim, Jaehyun Ha, George Fletcher, and Wook-Shin Han[1]. 2023. Guaranteeing the $\tilde{O}(\frac{AGM}{OUT})$ Runtime for Uniform Sampling and Size Estimation over Joins. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS '23)*, June 18–23, 2023, Seattle, WA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3584372.3588676>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PODS '23, June 18–23, 2023, Seattle, WA, USA.

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0127-6/23/06...\$15.00
<https://doi.org/10.1145/3584372.3588676>

1 INTRODUCTION

The evaluation of join queries is one of the most fundamental problems in databases [1, 29–32, 34]. In theory, reducing the time complexity of evaluation algorithms has been the main goal [3, 19, 31, 33]. This also applies to the counting problem [25], which we study in this paper. Given a join query Q and a database D , an answer of Q in D is a mapping from free variables of Q to attribute values in D (or inversely as stated in [16]) constrained by the relations in Q . For example, if a database D of a social network has a binary relation $R(A, B)$ of friendship and a tuple $(a, b) \in R(A, B)$ indicates that two people a, b are friends, finding all triples (a, b, c) of people who are all friends, is equivalent to finding the answers of the triangle join query $R(B, C)$ and $R(A, C)$ are renamed relations of $R(A, B)$:

$$Q(A, B, C) = R(A, B) \wedge R(B, C) \wedge R(A, C) \quad (1)$$

The complexity of evaluating join queries is commonly expressed as $\tilde{O}(IN^w + OUT)$ where IN is the input size and $w(\geq 1)$ is a width of Q (see §2 for definition of IN and example widths). While acyclic queries can be evaluated with $w = 1$ [33], cyclic queries have higher w values, i.e., more difficult to solve. If w is fractional edge cover number ρ (see Definition 1), IN^w becomes AGM , the AGM bound [7] which is an upper bound of OUT. Instead of evaluating join queries, the counting problem is to compute $|Q|$, the join size or the number of answers (e.g., triples above). It has its own application for answering COUNT queries in DBMS. Then, the complexity can be reduced from $\tilde{O}(IN^w + OUT)$ to $\tilde{O}(IN^w)$ [25].

Approximate counting problem is to approximate $|Q|$. Approximate algorithms are alternatively used whenever the exact counting is expensive and approximation is enough. In practice, a famous application is cost-based query optimization which requires efficient approximation of thousands of sub-queries [28]. In theory, going beyond the polynomial time algorithms, approximate algorithms established the complexity inversely proportional to OUT, under the requirement that guarantees an arbitrary small error with a high probability (see §2 for formal definitions). In a limited setting where every relation in Q is binary and identical (e.g., as in (1)), the complexity of $\tilde{O}(IN^w)$ has reduced to $\tilde{O}(\frac{IN^w}{OUT})$ [6, 15]. However, in a general setting where Q contains higher-arity relations, the complexity has reduced to $\tilde{O}(\frac{IN^{w+1}}{OUT})$ or $\tilde{O}(\frac{IN^w}{OUT} + IN)$ only [11], with

*corresponding author

an additional multiplicative or additive factor of IN. Furthermore, all these methods have $w = \rho$ only (i.e., $\tilde{O}(\frac{IN^w}{OUT}) = \tilde{O}(\frac{AGM}{OUT})$), and are not extended to other widths such as *fractional hypertree width* (see Definition 3) smaller than ρ . In this paper, we propose a new method that achieves $\tilde{O}(\frac{AGM}{OUT})$ complexity for Q containing any-arity relations, under the same $w = \rho$. As a corollary, ours achieve a linear or even constant complexity for a sufficiently large OUT. We also extend ours to incorporate fractional hypertree width by using generalized hypertree decompositions (GHDs), achieving a lower complexity than $\tilde{O}(\frac{AGM}{OUT})$ when OUT is small.

A closely related problem to approximate counting is uniform sampling, which has its own important applications such as training set generation for machine learning [34]. Our method also supports uniform sampling over joins. We define the problems and technical concepts in §2, explain related work in §3 and overview of our results in §4. We present our method of size estimation and uniform sampling over joins in §5 including a unified framework of previous work, extend our method using GHDs in §6, propose optimization techniques in §7 and future work in §8. More details of existing algorithms and extension of our algorithms to join-project (a.k.a. conjunctive) queries are presented in Appendix.

2 TECHNICAL BACKGROUND

Hypergraph. A join query Q is commonly represented as a *hypergraph* $\mathcal{H}(\mathcal{V}, \mathcal{E})$, where *hypernodes* $\mathcal{V}$ is the set of variables in Q and *hyperedges* $\mathcal{E} \subseteq 2^{\mathcal{V}}$ [31]. Each hyperedge $F \in \mathcal{E}$ is a subset of $\mathcal{V}$ and specifies the relation $R(F)$ (or R_F for brevity) in Q . For example, the query in (1) has $\mathcal{V} = \{A, B, C\}$ and $\mathcal{E} = \{\{A, B\}, \{B, C\}, \{A, C\}\}$. For any $I \subseteq \mathcal{V}$, we define two notations used throughout the paper: $\mathcal{E}_I := \{F \in \mathcal{E} \mid F \cap I \neq \emptyset\}$ and $\mathcal{E}_{I,\pi} := \{\pi_I F \mid F \in \mathcal{E}_I\}$, i.e., the set of hyperedges in $\mathcal{E}$ that intersect with I and the projection of each edge in $\mathcal{E}_I$ onto I .

In relational algebra, Q is equivalent to $\bowtie_{F \in \mathcal{E}} R_F = R_{F_1} \bowtie \dots \bowtie R_{F_{|\mathcal{E}|}}$. Here, join ($\bowtie$) is a *commutative* binary operator between relations, allowing us to use $\bowtie_{F \in \mathcal{E}}$. Table 1 summarizes the operators used in this paper.

Table 1: Operators used in the paper.

$\bowtie$	natural join operator
$\ltimes$	semi-join operator
π	projection operator
σ	selection operator
$\wr$	list concatenation operator

Since all variables in $\mathcal{V}$ are free (output) variables in join queries, join queries are subsumed by *full* conjunctive queries. For a more general class of join-project (a.k.a. conjunctive) queries, the set of output variables is a subset of $\mathcal{V}$. We assume Q is a join query and extend to join-project queries in §C.

Input and output size. We denote $IN(\mathcal{H}, D) = \max_{F \in \mathcal{E}} |R_F|$ as the input size ($|R_F|$ is the number of tuples in R_F) and $OUT(\mathcal{H}, D) = |Q|$, the number of query answers, as the output size of Q (or $\mathcal{H}$) in D . We drop $\mathcal{H}$ and D if the context is clear, as well as in upcoming notations.

Complexity of join. The time complexity of join evaluation algorithms is commonly expressed as $\tilde{O}(IN^w + OUT)$ where w is a *width* of Q , and $\tilde{O}$ hides a polylogarithmic factor of IN [31]. We assume

that Q is extremely small compared to D and regard $|\mathcal{V}|$ and $|\mathcal{E}|$ as constants as in [6, 11, 31]. *Worst-case optimal* (WCO) algorithms such as GENERICJOIN [31] (Algorithm 4) achieve $w = \rho(\mathcal{H}, D)$, called *fractional edge cover number*:

Definition 1. [31]: *The fractional edge cover number $\rho(\mathcal{H}, D)$ is the optimal solution of the linear program (LP): $\min_{\{x_F \mid F \in \mathcal{E}\}} \sum_{F \in \mathcal{E}} x_F \cdot \log_{IN} |R_F|$ s.t. $\sum_{F \ni v} x_F \geq 1 : \forall v \in \mathcal{V}$ and $0 \leq x_F \leq 1 : \forall F \in \mathcal{E}$.*

It is well known that $\prod_{F \in \mathcal{E}} |R_F|^{x_F}$ is an upper bound of OUT for any fractional edge cover x satisfying the constraints of the LP [7, 17]. The optimal value, $IN^\rho = \min_x \prod_{F \in \mathcal{E}} |R_F|^{x_F}$, is called the AGM bound of $\mathcal{H}$ on D denoted as $AGM(\mathcal{H}, D)$ [7]. For example, the query in (1) has $IN = |R_{\{A,B\}}| = |R_{\{B,C\}}| = |R_{\{A,C\}}|$ and $\rho = 3/2$ from the optimal solution $x_{\{A,B\}} = x_{\{B,C\}} = x_{\{A,C\}} = 1/2$ for the LP in Definition 1. Therefore, its AGM bound is $IN^\rho = IN^{3/2}$. **Generalized hypertree decomposition.** As a special case, the YANNAKAKIS algorithm [33] achieves $w = 1$ if Q is acyclic. It builds a *join tree* by assigning each hyperedge $F \in \mathcal{E}$ to a tree node and performs dynamic programming (DP)-like join over the nodes' outputs (i.e., R_F 's) in a bottom-up manner.

To bridge the gap between $w = \rho$ and $w = 1$ when Q is cyclic, generalized hypertree decompositions (GHDs, see Definition 2) [20] extend the tree for arbitrary joins by assigning multiple hyperedges to a tree node, where the DP-like join is performed over these nodes' outputs (i.e., answers of the corresponding sub-query) computed by WCO algorithms. This way, w decreases from ρ to *fractional hypertree width* $fhtw$ (see Definition 3). We refer to a comprehensive survey [31] for more details about these concepts and §A for our explanations of algorithms.

Definition 2. [20]: *A GHD of $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ is a pair $(\mathcal{T}, \chi)$ where 1) $\mathcal{T}$ is a tree of $(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}})$, 2) $\chi : \mathcal{V}_{\mathcal{T}} \rightarrow 2^{\mathcal{V}}$, 3) for each $F \in \mathcal{E}$, there exists $t \in \mathcal{V}_{\mathcal{T}}$ s.t. $F \subseteq \chi(t)$, and 4) for each $v \in \mathcal{V}$, $\{t \in \mathcal{V}_{\mathcal{T}} \mid v \in \chi(t)\}$ forms a non-empty connected subtree of $\mathcal{T}$.*

Here, $\chi(t)$ is called the *bag* of t , and 4) is called the *running intersection property*. Each node t corresponds to a sub-hypergraph $\mathcal{H}_{\chi(t)}$ of $\mathcal{H}$ where $\mathcal{H}_{\chi(t)} := (\chi(t), \mathcal{E}_{\chi(t),\pi})$. We then define the fractional hypertree width [22].

Definition 3. [22]: *The fractional hypertree width 1) of a GHD $(\mathcal{T}, \chi)$ of $\mathcal{H}$, $fhtw(\mathcal{T}, \mathcal{H})$, is defined as $\max_{t \in \mathcal{V}_{\mathcal{T}}} \rho(\mathcal{H}_{\chi(t)})$, and 2) of $\mathcal{H}$, $fhtw(\mathcal{H})$, is defined as $\min_{(\mathcal{T}, \chi)} fhtw(\mathcal{T}, \mathcal{H})$.*

Given a GHD $(\mathcal{T}, \chi)$, GHDJOIN [25] performs GENERICJOIN for each $\mathcal{H}_{\chi(t)}$ ($t \in \mathcal{V}_{\mathcal{T}}$) and YANNAKAKIS [33] on these results along the join tree (see Algorithms 5-7 in §A.2-A.3). Since YANNAKAKIS runs in linear - $\tilde{O}(IN + OUT)$ - time for α -acyclic join queries [9], the total runtime of GHDJOIN is $\tilde{O}(IN^{fhtw(\mathcal{T}, \mathcal{H})} + OUT)$, since the input size for YANNAKAKIS is $\tilde{O}(IN^{\max_{t \in \mathcal{V}_{\mathcal{T}}} \rho(\mathcal{H}_{\chi(t)})})$ after GENERICJOIN. It is sufficient to execute a brute-force algorithm to find $fhtw(\mathcal{H})$ since the number of possible GHDs is bounded by the query size.

Counting. For the counting problem, the complexity is reduced to $\tilde{O}(IN^w)$ without the +OUT term. This is a direct result of Joglekar et al. [25], solving a more general problem of evaluating aggregation queries (e.g., count, sum, max). We explain in detail in §6.1.

Approximate counting. Approximate counting achieves even smaller complexities. The primary goal of approximate counting, especially using sampling, is to obtain an arbitrarily small error with a high probability, formally defined as [6, 11]:

Definition 4. [6]: For a given $\mathcal{H}$, error bound $\epsilon \in (0, 1)$, and probability threshold $\delta \in (0, 1)$, if $P(|Z - OUT| \geq \epsilon OUT) \leq \delta$ for a random variable Z approximating OUT , then Z is a $(1 \pm \epsilon)$ -approximation of OUT .

The challenge is, how far can we reduce the complexity while achieving the above goal. Assume that we approximate OUT using an unbiased estimator with a random variable Y , i.e., $\mathbb{E}[Y] = OUT$, where $\mathbb{E}[Y]$ is the expectation of Y . Assume that the variance $\text{Var}[Y]$ of Y is bounded and an upper bound U_{var} is known, and the time complexity of instantiating (or computing) Y is upper-bounded by U_{time} . Then, we can trade off the approximation accuracy and efficiency by controlling the number of runs N for sampling. Formally, let $Z = (Y_1 + Y_2 + \dots + Y_N)/N$ where every Y_i ($1 \leq i \leq N$) is an equivalent random variable to Y . Then, $\mathbb{E}[Z] = \mathbb{E}[Y] = OUT$ and $\text{Var}[Z] = \text{Var}[Y]/N$. Hence, by setting $N = \frac{U_{var}}{\epsilon^2 \delta OUT^2}$, we can achieve the following from the Chebyshev's inequality:

$$P(|Z - OUT| \geq \epsilon OUT) \leq \frac{\text{Var}[Z]}{\epsilon^2 \mathbb{E}[Z]^2} = \frac{\text{Var}[Y]}{N \epsilon^2 OUT^2} \leq \frac{U_{var}}{N \epsilon^2 OUT^2} = \delta \quad (2)$$

Since the complexity of computing each Y_i is bounded by U_{time} , the complexity of computing Z is bounded by $N \cdot U_{time} = \frac{U_{var} U_{time}}{\epsilon^2 \delta OUT^2}$. Therefore, the key challenge is to implement an unbiased estimator with $U_{var} U_{time}$ as smallest as possible. ϵ and δ are regarded as constants as in $\tilde{O}$ [6, 11]. One can easily replace $\frac{1}{\delta}$ in N with $\log \frac{1}{\delta}$ using the median trick [24].

Uniform sampling. If every query answer has the same probability p to be sampled, then the sampler is a uniform sampler [11, 15]. Since there are $|Q| = OUT$ answers, p can be at most $1/OUT$. If $p < 1/OUT$, the sampling has the probability to fail (to sample any of the query answers) which is $1 - p \cdot OUT$.

For acyclic queries, uniform sampling can be done in $\tilde{O}(1)$ time after $\tilde{O}(IN)$ -time preprocessing [34], by additionally storing the intermediate join sizes of tuples during the bottom-up DP in YANNAKAKIS (see §A.2 for more details).

3 RELATED WORK

In the context of approximate counting (and uniform sampling, which is closely related), there have been two lines of work: 1) determining the classes of queries of tractable and intractable cases and 2) reducing the degree of polynomial-time complexity for tractable cases, especially for join or join-project queries.

The first class of work widens the class of queries that admit fully polynomial-time randomized approximation scheme (FPRAS) and fully polynomial-time almost uniform sampler (FPAUS) [5]. Arenas et al. [5] showed that every class of conjunctive queries with bounded hypertree width admits FPRAS and FPAUS. Focke et al. [16] extended this to conjunctive queries with disequalities and negations, under some relaxations from FPRAS. These works focus on showing the existence of a polynomial-time algorithm over a wider class of queries, but not on reducing the exact degree of the polynomial complexity.

The second class of work [4, 6, 8, 11, 14, 15] focuses on reducing the degree for a specific range of queries, and even further to making the complexity inversely proportional to OUT , e.g., $\tilde{O}(\frac{IN^w}{OUT})$. However, they all consider $w = \rho$ only (i.e., $\tilde{O}(\frac{IN^w}{OUT})$ is $\tilde{O}(\frac{AGM}{OUT})$), and the complexity $\tilde{O}(\frac{AGM}{OUT})$ has been achieved for limited classes of queries. Assadi et al. [6] proposed a Simple Sublinear-Time Estimator (SSTE for short) with the complexity of $\tilde{O}(\frac{AGM}{OUT})$ for unlabeled graph queries, i.e., where Q contains identical binary relations only as in the query in (1). However, we found that a missing factor in the proof by Assadi et al. [6] prevented SSTE from satisfying the $\tilde{O}(\frac{AGM}{OUT})$ bound and thus notified the author to modify the proof (see §E in our full paper [26]). They also proposed some conjectures on labeled graphs (i.e., removing the identity assumption), but 1) no concrete algorithms were given and 2) the proposed bound for labeled graphs is larger than $\tilde{O}(\frac{AGM}{OUT})$ (see §D.2). Therefore, it is not known whether these methods can be extended to labeled graphs with $\tilde{O}(\frac{AGM}{OUT})$ time. Fichtenberger et al. [15] extended SSTE to sampling Subgraphs Uniformly in Sublinear Time (SUST for short) for uniform sampling while achieving the same complexity. Here, they said sublinear from the assumption that OUT is large as $\Omega(IN^{\rho-1})$. Unlike join processing, it is unnecessary to scan the inputs for each query that requires $\tilde{O}(IN)$ time.

Kim et al. [27] proposed ALLEY, a hybrid method that combines sampling and synopsis. We analyze the sampling part of ALLEY since analyzing the synopsis is out of scope. ALLEY solves the approximate subgraph counting problem (but not uniform sampling) for labeled graphs, where the relations in Q are binary but not necessarily identical. The complexity, however, is $\tilde{O}(AGM)$ instead of $\tilde{O}(\frac{AGM}{OUT})$, which we explain why in §5.

For a more general setting where relations in Q can have higher arities, Chen & Yi [11] proposed GJ-SAMPLE that also performs uniform sampling, and achieved $\tilde{O}(\frac{IN \cdot AGM}{OUT})$ complexity. The additional factor of IN compared to SSTE and SUST results from computing sample probabilities (or weights) prior to sampling, where the number of candidates is $\tilde{O}(IN)$. We explain in detail in §5. Therefore, they do not achieve $\tilde{O}(\frac{AGM}{OUT})$ complexity for arbitrary join queries and leave it as an open problem [11]. Note that it is important to remove the IN factor, for IN can be higher than $\frac{AGM}{OUT}$ when relation sizes are highly skewed. For example, for a triangle query $\mathcal{H}$ with $\mathcal{E} = \{F_1, F_2, F_3\}$, $|R_{F_1}| = N^{10}$, $|R_{F_2}| = |R_{F_3}| = N$, and $OUT = N$, then IN is N^{10} and $\frac{AGM}{OUT}$ is $\frac{N^2}{N} = N$ for $\rho = \frac{1}{5}$.

We notice that Deng et al. [12] have independently pursued the same primary objective as ours, which is to attain $\tilde{O}(\frac{AGM}{OUT})$ for uniform sampling and size estimation over joins. Both papers have been accepted in this PODS. The authors recursively 1) partition the input space into a constant number of sub-spaces and 2) sample a sub-space, to reduce the number of sample candidates from $\tilde{O}(IN)$ to $\tilde{O}(1)$. Our approach differs from theirs since we do not compute probabilities for candidates prior to sampling nor modify the set of candidates. Instead, we exploit known probability distributions to sample a candidate.

4 OUR RESULTS

We first analyze existing sampling-based approximation algorithms in a new unified framework in §5. We present a new algorithm that bounds $U_{var} U_{time}$ by $\tilde{O}(AGM \cdot OUT)$ for arbitrary queries for

the first time, achieving the complexity of $\tilde{O}(\frac{AGM}{OUT})$ (see §2). To avoid pre-sampling overheads in GJ-SAMPLE, we propose *degree-based rejection sampling* (DRS) that first samples a sample space from a *meta* sample space and then samples a value from the sampled space, where the value can be *rejected* based on its *degree*. We explain the details in §5. The results of Chen & Yi then directly hold for DRS. The removed overhead reduces the complexity down to $\tilde{O}(\frac{AGM}{OUT})$ for arbitrary joins. We also extend DRS using GHDs in §6. The following theorems state our results which we prove in the subsequent sections.

Theorem 1. *DRS performs $(1 \pm \epsilon)$ -approximation of OUT in $\tilde{O}(\frac{AGM}{OUT})$ time.*

Theorem 2. *If OUT is small enough, DRS with GHDs can perform $(1 \pm \epsilon)$ -approximation of OUT with a lower complexity than $\tilde{O}(\frac{AGM}{OUT})$. A sufficient condition is when OUT is $\tilde{O}(IN^{p-fhtw})$.*

Additionally, we extend ALLEY to support arbitrary join queries in §5 and a core lemma of SSTE/SUST to hold for labeled graphs in §D. We discuss our extension to join-project (a.k.a. conjunctive queries) in §C and extension of the $\tilde{O}(1)$ -time uniform sampler [34] in §2 to support cyclic queries using our framework in §A.2.

5 ACHIEVING $\tilde{O}(\frac{AGM}{OUT})$ BOUND

5.1 Generic Sampling-based Framework

We can classify existing sampling-based methods into three groups: variable-at-a-time (a.k.a., vertex-at-a-time) [11, 27], edge-at-a-time [29], and component-at-a-time [6, 15]. The first/second group samples a variable/hyperedge at a time until every variable/hyperedge in $\mathcal{H}$ is sampled, while the third group samples a component, either a cycle with an odd number of edges or a star, at a time, having a larger sample granularity than the other two groups.

To analyze these groups in a unified manner, we propose a powerful sampling-based estimation framework GENERICCARDEST (Algorithm 1). It returns an unbiased estimate of $OUT(\mathcal{H}_s)$, where $\mathcal{H}_s$ is the *residual hypergraph* given a current sample s which is initially empty. Formally, $\mathcal{H}_s := \bowtie_{F \in \mathcal{E}_O} \pi_O(R_F \ltimes s)$ and $\mathcal{H}_\emptyset = \mathcal{H}$ for $s = \emptyset$.

Algorithm 1. GENERICCARDEST($\mathcal{H}(\mathcal{V}, \mathcal{E}), O, s$)

Input: Query hypergraph $\mathcal{H}$, variables O to sample, and current sample s

```

1: if ( $O = \emptyset$ ) then
2:   return 1
3:  $I \leftarrow \text{GETNONEMPTYSUBSET}(O) \text{ /* } I \neq \emptyset \text{ */}$ 
4:  $\Omega_I, k \leftarrow \text{GETSAMPLESPACEANDSIZE}(I, \mathcal{H}, s)$ 
5:  $P \leftarrow \text{GETDISTRIBUTION}(\Omega_I) \text{ /* } P(\perp) = 1 - \sum_{s_I \in \Omega_I} P(s_I) \text{ */}$ 
6:  $S_I \leftarrow \text{SAMPLEFROM}(\Omega_I, k, P) \text{ /* } |S_I| = k \text{ */}$ 
7:  $S_I, k \leftarrow |S_I|, \{P(s_I) | s_I \in S_I\} \leftarrow \text{REJECTANDADJUST}(S_I, P, \mathcal{H}, s)$ 
8: return  $\frac{1}{k} \sum_{s_I \in S_I} \frac{1}{P(s_I)} \cdot \mathbb{I}[s_I \in \bowtie_{F \in \mathcal{E}_I} \pi_I(R_F \ltimes s)] \cdot \text{GENERICCARDEST}(\mathcal{H}, O \setminus I, s \cup s_I)$ 

```

An invariance is that $s \in \bowtie_{F \in \mathcal{E}_{V \setminus O}} \pi_{V \setminus O} R_F$, i.e., s is an answer to the sub-query of Q on the *bound* variables $V \setminus O$. This is a key to prove Lemma 1. Due to space limitations, we prove all lemmas and propositions in §E of our full paper [26].

Lemma 1. *GENERICCARDEST returns an unbiased estimate of $|\bowtie_{F \in \mathcal{E}} R_F|$ for $O = V$ and $s = \emptyset$.*

We now explain each line of Algorithm 1. If there is no variable to sample for, just return one in Lines 1-2. Otherwise, Line 3 takes a subset I of O to sample at this step. I is 1) a singleton set for variable-at-a-time methods, 2) an edge or its projection for edge-at-a-time methods, or 3) a part of a component for component-at-a-time methods. Line 4 defines the *sample space* Ω_I and *sample size* k . Line 5 computes P , a probability distribution over $\Omega_I \cup \{\perp\}$; $\perp$ is a null tuple that does not belong to any join answer. Line 6 samples s_I from Ω_I where $|s_I| = k$. Line 7 optionally rejects samples in S_I . Here, S_I, k , and $\{P(s_I) | s_I \in S_I\}$ are adjusted according to the accepted samples; $P(s_I)$ is multiplied by the probability of accepting s_I . GENERICCARDEST in Line 8 returns an unbiased estimate of the residual query $\mathcal{H}_{s \cup s_I}$. The unbiasedness of the final return value at Line 8 is guaranteed by the inverse probability factor, $\frac{1}{P(s_I)} \mathbb{I}[\cdot]$, which is the key idea of the Horvitz-Thompson estimator [23]. $\mathbb{I}[c]$ denotes the indicator which is 1 if c is true and 0 otherwise.

5.2 Query Model

GENERICCARDEST relies on a certain query model used in previous WCO join and sampling algorithms [6, 11, 30–32], following the property testing model [18]. To avoid any confusion with the join query Q , we refer to the queries here as operations. For a relation R_F , the query model allows that $\pi_I(R_F \ltimes s)$ in GENERICCARDEST can be readily obtained in $\tilde{O}(1)$ time. We explain the underlying index structures that enable this later in §5.6 for brevity.

The same indexes provide the following $\tilde{O}(1)$ -time operations as well. Here, T can be a projection/selection of another relation, which is also a relation.

- Degree(T, s): $|T \ltimes s|$ for a relation T and a tuple s
- Access(T, I, i): i -th element of $\pi_I(T)$ for a relation T and attributes I
- Exist(T, r): test whether or not a row r exists in T
- Sample(T): uniformly sample a tuple r from T

Eden et al. [13] have developed their algorithms using operations on binary tables. The above four operations are generalizations of theirs to n -ary tables.

5.3 State-of-the-art Sampling-based Estimators

We explain state-of-the-art sampling-based estimators as instances of GENERICCARDEST.

WANDERJOIN [29] is a practical edge-at-a-time method. The edges in $\mathcal{E}$ are first ordered by an index $i \in [1, |\mathcal{E}|]$. At the i -th step, $I = F_i \cap O$, $\Omega_I = \pi_I(R_{F_i} \ltimes s)$, $k = 1$, and $P(s_I) = \frac{1}{|\Omega_I|}$. That is, one tuple s_I is uniformly sampled from Ω_I . Note that O can reduce to $\emptyset$ before proceeding to the last edge, e.g., the triangle query. However, [29] proceeds first with sampling for every edge and then checks the join predicates, resulting in unnecessary computations.

ALLEY+. ALLEY [27] is a variable-at-a-time method (i.e., $|I| = 1$) and assumes that every R_F is binary. We here explain ALLEY+, our extension of ALLEY to n -ary relations: $\Omega_I = \cap_{F \in \mathcal{E}_I} \pi_I(R_F \ltimes s)$, $k = \lceil b \cdot |\Omega_I| \rceil$ for some fixed $b \in (0, 1]$, and $P(s_I) = \frac{1}{|\Omega_I|}$. Here, the sampling is done *without* replacement. Due to the heavy intersection ($\cap$) operation, the runtime of ALLEY+ is $\tilde{O}(AGM(\mathcal{H}))$.

(Proposition 6). At an extreme case of $b = 1$, ALLEY+ becomes an instance of GENERICJOIN.

SSTE [6] and **SUST** [15] are component-at-a-time methods. Assuming that every R_F is binary and identical, Lemma 8 in §D decomposes $\mathcal{H}$ into a set of components, where each component is either an odd cycle or a star. SSTE and SUST slightly differ in sampling odd cycles and stars, and SUST performs uniform sampling over joins. We explain the details in §D due to space limitations.

GJ-SAMPLE [11] is a variable-at-a-time method (i.e., $|I| = 1$). Here, $\Omega_I = \pi_I(R_{FGJ} \bowtie s)$ where $F^{GJ} = \arg \min_{F \in \mathcal{E}_I} |\pi_I(R_F \bowtie s)|$. This avoids the intersection operation in ALLEY+, allowing GJ-SAMPLE to achieve $\tilde{O}(\frac{AGM(\mathcal{H})}{OUT})$ runtime for sampling instead of $\tilde{O}(AGM(\mathcal{H}))$ [11]. $k = 1$ and $P(s_I) = \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ where $AGM(\mathcal{H}_s) = \prod_{F \in \mathcal{E}_Q} |R_F \bowtie s|^{x_F}$ and $AGM(\mathcal{H}_{s \cup s_I}) = \prod_{F \in \mathcal{E}_{Q \setminus I}} |R_F \bowtie (s \cup s_I)|^{x_F}$, where x is a fractional edge cover of the original query $\mathcal{H}$ [11]. Then, due to Lemma 7 in §B, $\sum_{s_I \in \Omega_I} P(s_I) \leq 1$.

More importantly, setting $P(s_I) = \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ enables GJ-SAMPLE to perform uniform sampling over Q , with $\frac{1}{AGM(\mathcal{H})}$ probability for each answer of Q . Let s be a final sample that reaches Line 2, and $\mathcal{H}_{s_1}, \mathcal{H}_{s_2}, \dots, \mathcal{H}_{s_n}$ be the series of hypergraphs $\mathcal{H}_{s \cup s_I}$ at Line 8. Then, $AGM(\mathcal{H}_{s_n}) = 1$ from $\mathcal{E}_Q = \mathcal{E}_\emptyset = \emptyset$ for $\mathcal{H}_{s_n}$, and

$$P(s) = \frac{AGM(\mathcal{H}_{s_1})}{AGM(\mathcal{H})} \cdots \frac{AGM(\mathcal{H}_{s_n})}{AGM(\mathcal{H}_{s_{n-1}})} = \frac{AGM(\mathcal{H}_{s_n})}{AGM(\mathcal{H})} = \frac{1}{AGM(\mathcal{H})}. \quad (3)$$

Since $|Q| = OUT$, a call to GENERICCARDEST succeeds to sample any answer of Q with $\frac{OUT}{AGM(\mathcal{H})}$ probability and fails with $1 - \frac{OUT}{AGM(\mathcal{H})}$ probability. Note that if $P(s_I)$ is set to $OUT(\mathcal{H}_{s \cup s_I})/OUT(\mathcal{H}_s)$, then $P(s) = \frac{1}{OUT}$, so every call to GENERICCARDEST will succeed. However, computing $OUT(\mathcal{H}_{s \cup s_I})$ for every $s_I \in \Omega_I$ is intractable, which is the reason for GJ-SAMPLE to use $\frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ that can be computed in $\tilde{O}(1)$ time.

However, computing $\frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ values at Line 5 takes $\tilde{O}(|\Omega_I|) = \tilde{O}(|\pi_I(R_{FGJ} \bowtie s)|) = \tilde{O}(IN)$ time. This results in an IN factor in GJ-SAMPLE's runtime, $\tilde{O}(\frac{IN \cdot AGM(\mathcal{H})}{OUT})$ [11]. In order to remove this IN factor, Chen & Yi [11] separated out *sequenceable* queries, where all $P(s_I)$ values required in the sampling procedure can be precomputed in $\tilde{O}(IN)$ time. This results in $\tilde{O}(IN + \frac{AGM(\mathcal{H})}{OUT})$ runtime for sequenceable queries and still $\tilde{O}(\frac{IN \cdot AGM(\mathcal{H})}{OUT})$ for non-sequenceable queries. Determining whether a query is sequenceable or not requires a brute-force algorithm of $\tilde{O}(1)$ time, since the search space is bounded by the query size [11].

5.4 Beyond the State of the Art with Degree-based Rejection Sampling

We propose *degree-based rejection sampling* (DRS) to avoid computing $\frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ values in GJ-SAMPLE while achieving a similar $P(s_I)$, only a constant factor smaller. While we set $|I| = 1$ and $k = 1$ as in GJ-SAMPLE, we sample F^* (a counterpart of F^{GJ} in GJ-SAMPLE) uniformly from $\mathcal{E}_I$ and let $\Omega_I = \pi_I(R_{F^*} \bowtie s)$. Therefore, $\{\pi_I(R_F \bowtie s) \mid F \in \mathcal{E}_I\}$ is our *meta* sample space. For each $s_I \in \Omega_I$ and $F \in \mathcal{E}_I$, we define the *relative degree* of s_I in $R_F \bowtie s$ as $rdeg_{F,s}(s_I) := \frac{|R_F \bowtie (s \cup s_I)|}{|R_F \bowtie s|}$.

In order to use $P(s_I) = rdeg_{F^*,s}(s_I)$ in Line 5, we 1) uniformly sample a row t from $R_{F^*} \bowtie s$ and 2) let $s_I = \pi_I t$ in Line 6, without computing $P(s_I)$ for every $s_I \in \Omega_I$. Then, $rdeg_{F,s}(s_I)$ for every $F \in \mathcal{E}_I$ is computed, and s_I is *rejected* if $F^* \neq \arg \max_{F \in \mathcal{E}_I} rdeg_{F,s}(s_I)$ (Line 7). We first assume that one F has a higher $rdeg_{F,s}(s_I)$ than all other edges in $\mathcal{E}_I$. Then, every $P(s_I)$ is multiplied by $\frac{1}{|\mathcal{E}_I|}$ at Line 7, resulting in $P(s_I) = \frac{1}{|\mathcal{E}_I|} rdeg_{F^*,s}(s_I)$. Line 7 further uses $p = \frac{AGM(\mathcal{H}_{s \cup s_I})}{rdeg_{F^*,s}(s_I) \cdot AGM(\mathcal{H}_s)} (\leq 1 \text{ from Lemma 2})$ as the keeping probability of s_I to make the final $P(s_I) = \frac{1}{|\mathcal{E}_I|} \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$.

Lemma 2. $\frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)} \leq \max_{F \in \mathcal{E}_I} rdeg_{F,s}(s_I) = rdeg_{F^*,s}(s_I)$ if $s_I \in \pi_I(R_{F^*} \bowtie s)$.

From (3), any final sample s that reaches Line 2 has $P(s) = \frac{1}{AGM(\mathcal{H})} \prod_{I \subset \mathcal{V}, |I|=1} \frac{1}{|\mathcal{E}_I|}$ probability to be sampled, indicating a uniform sampling over Q . Note that the added term $\prod_I \frac{1}{|\mathcal{E}_I|}$ is regarded as a constant since it depends on the query size only. If we break our initial assumption so that $m > 1$ edges in $\mathcal{E}_I$ can have the same maximum relative degree, the $P(s_I)$ above will increase by m times. Therefore, we simply decrease the keeping probability p by m times to preserve $P(s_I) = \frac{1}{|\mathcal{E}_I|} \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$.

Example 1. Let N and M be two arbitrary numbers. Let $R(A, B) = \{(i, 1) \mid i \in [1, M]\} \cup \{(1, j) \mid j \in [2, N]\}$ be a binary relation and $T(A, C)$ is obtained by renaming B to C in R . Suppose we have a join query $Q(A, B, C) = R(A, B) \bowtie T(A, C)$. For ease of explanation, we use a hyperedge F and its corresponding relation R_F interchangeably. Then, we have an optimal fractional edge cover of Q as $x_R = x_T = 1$. We choose $I = \{A\}$, $I = \{B\}$, and $I = \{C\}$ in turn for DRS. For $I = \{A\}$ and $s = \emptyset$, $\mathcal{E}_I = \{R, T\}$, so we sample a relation from $\mathcal{E}_I$. Assume that R is sampled, and then a row $t = (1, 1)$ is sampled from R . Then, $s_I = 1$ and $rdeg_{R,s}(s_I) = \frac{N}{N+M-1} = rdeg_{T,s}(s_I)$, so R is not rejected. Since R and T tie w.r.t. s_I , we keep s_I with probability $p = \frac{1}{2} \frac{1}{rdeg_{R,s}(s_I)} \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)} = \frac{1}{2} \frac{1}{rdeg_{R,s}(s_I)} \frac{N^2}{(N+M-1)^2}$. Then, $P(s_I) = \frac{1}{2} \frac{1}{rdeg_{R,s}(s_I)} \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)} = \frac{1}{2} \frac{1}{rdeg_{R,s}(s_I)} \frac{N^2}{(N+M-1)^2} = \frac{1}{2} \frac{N^2}{(N+M-1)^2}$. Next, for $I = \{B\}$ and $s = \{A = 1\}$, $\mathcal{E}_I = \{R\}$, and $rdeg_{R,s}(s_I) = \frac{1}{N}$ for any $s_I \in \pi_I(R \bowtie s)$. Then, $P(s_{\{B\}}) = \frac{1}{|\mathcal{E}_I|} \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)} = \frac{N}{N^2} = \frac{1}{N}$. Similarly, $P(s_{\{C\}}) = \frac{1}{N}$ for $I = \{C\}$. In total, a final sample s that reaches Line 2 of GENERICCARDEST has $P(s) = P(s_{\{A\}})P(s_{\{B\}})P(s_{\{C\}}) = \frac{1}{2} \frac{1}{(N+M-1)^2} = \prod_I \frac{1}{|\mathcal{E}_I|} \frac{1}{AGM(\mathcal{H})}$.

5.5 Unified Analysis

This section analyzes the bounds of variance and runtime (U_{var} and U_{time} in §2) of sampling-based estimators in a unified manner. Note that Lemma 1 already states the unbiasedness of estimators. Theorem 1 can be proved from Propositions 4 and 8; $U_{var}U_{time}$ is $\tilde{O}(AGM \cdot OUT)$.

We first define two random variables, 1) $Z_{\mathcal{H}_s}$ for the output of GENERICCARDEST given s and 2) $Z = \sum_{1 \leq i \leq N} \frac{Z_{\mathcal{H}_s}^i}{N}$ for our final estimate. Here, $Z_{\mathcal{H}_s}^i$'s are independent and identical to $Z_{\mathcal{H}} = Z_{\mathcal{H}_\emptyset}$, and N is the number of initial calls to GENERICCARDEST with $s = \emptyset$. Let $T_{\mathcal{H}_s}$ be the random variable for the number of core operations (including the four operations in §5.2) in GENERICCARDEST and T

for the total runtime of obtaining Z . Then, $\text{Var}[Z] = \frac{\text{Var}[Z_{\mathcal{H}}]}{N}$ and $\mathbb{E}[T] = N \cdot \mathbb{E}[T_{\mathcal{H}}]$.

Proposition 1. $\text{Var}[Z_{\mathcal{H}}] \leq 2^d \cdot \text{AGM}(\mathcal{H}) \cdot \text{OUT}$ for SSTE and SUST where d is the number of odd cycles and stars in $\mathcal{H}$.

Proposition 2. $\text{Var}[Z_{\mathcal{H}}] \leq \frac{t^{|V|}-t}{t-1} \cdot \text{OUT}^2$ for ALLEY+ where $t = \frac{2(1-b)}{b}$.

The upper bound U_{var} of ALLEY+ explains the unique property of ALLEY+, that $\text{Var}[Z_{\mathcal{H}}]$ approaches to 0 as b approaches to 1. That is, $\lim_{b \rightarrow 1} U_{\text{var}} = 0$ since $\lim_{b \rightarrow 1} t = 0$. In fact, this U_{var} is tighter than the original bound $U_{\text{org}} = \frac{b}{1-b} \left(\frac{1}{b^{|V|}} - 1 \right) \cdot \text{OUT}^2$ proved in [27], where $\lim_{b \rightarrow 1} U_{\text{org}} = \lim_{b \rightarrow 1} \frac{1+b+b^2+\dots+b^{|V|-1}}{b^{|V|-1}} \text{OUT}^2 = |V| \cdot \text{OUT}^2$.

Proposition 3. $\text{Var}[Z_{\mathcal{H}}] \leq |V| \cdot \text{AGM}(\mathcal{H}) \cdot \text{OUT}$ for GJ-SAMPLE.

Proposition 4. $\text{Var}[Z_{\mathcal{H}}] \leq |V| \cdot \prod_I |\mathcal{E}_I| \cdot \text{AGM}(\mathcal{H}) \cdot \text{OUT}$ for DRS. Note that $\prod_I |\mathcal{E}_I|$ is $\tilde{O}(1)$.

Proposition 5. $\mathbb{E}[T_{\mathcal{H}}]$ is $\tilde{O}(1)$ for SSTE.

Proposition 6. $\mathbb{E}[T_{\mathcal{H}}]$ is $\tilde{O}(b^{|V|} \text{AGM}(\mathcal{H}))$ for ALLEY+.

Proposition 7. For GJ-SAMPLE, $T_{\mathcal{H}}$ is $\tilde{O}(1)$ for sequenceable queries and $\tilde{O}(\text{IN})$ for non-sequenceable queries.

Proposition 8. $T_{\mathcal{H}}$ is $\tilde{O}(1)$ for SUST and DRS.

Until now, we have assumed that OUT is given in computing $N = \frac{U_{\text{var}}}{\epsilon^2 \delta \text{OUT}^2}$ before invoking GENERICCARDEST, which is unrealistic since our goal is to estimate OUT itself. To tackle this, we use the geometric search by Assadi et al. [6]. They first assume that OUT is large as $\text{AGM}(\mathcal{H})$ and run estimation with a small N [6]. Then, they perform a geometric search on OUT; assume a smaller OUT (decreasing OUT by 2), increase N , and run the estimation again. They repeat this until the assumed OUT becomes consistent with the algorithm output. While we implicitly assume that $\text{OUT} > 0$, we can detect when $\text{OUT} = 0$ in real applications whenever the assumed OUT falls below 1, in total $\tilde{O}(\text{AGM})$ time.

This geometric search adds a constant factor ($= 4$) in the $\tilde{O}$ bound of $T_{\mathcal{H}}$, not a log IN factor explained by Chen & Yi [11]. Starting with an assumption $1 \leq \frac{\text{AGM}(\mathcal{H})}{\text{OUT}} < 2$, assume that the geometric search stops at $2^k \leq \frac{\text{AGM}(\mathcal{H})}{\text{OUT}} < 2^{k+1}$. Then, the total sample size used so far is $\sum_{1 \leq i \leq k+1} \frac{2^i}{\epsilon^2 \delta} \leq \frac{2^{k+2}}{\epsilon^2 \delta} \leq \frac{4 \text{AGM}(\mathcal{H})}{\epsilon^2 \delta \text{OUT}}$, i.e., only a constant factor ($= 4$) is introduced.

Since GJ-SAMPLE can perform uniform sampling, it instead uses a simpler approach [11], by repeating the sampling until a constant number c of trials succeed. Then, the number of total trials becomes a random variable, having $\frac{\text{AGM}(\mathcal{H}) \cdot c}{\text{OUT}}$ as its expectation [11]. Therefore, $\mathbb{E}[T] = \tilde{O}(\text{IN} + \frac{\text{AGM}(\mathcal{H}) \cdot c}{\text{OUT}})$ for sequenceable queries and $\tilde{O}(\frac{\text{IN} \cdot \text{AGM}(\mathcal{H}) \cdot c}{\text{OUT}})$ for non-sequenceable queries. Using the same approach, DRS requires $\prod_I |\mathcal{E}_I| \frac{\text{AGM}(\mathcal{H}) \cdot c}{\text{OUT}}$ trials in expectation and thus $\mathbb{E}[T] = \tilde{O}(\frac{\text{AGM}(\mathcal{H})}{\text{OUT}})$, asymptotically faster than GJ-SAMPLE.

Finally, we give a simple explanation of why T (or $\mathbb{E}[T]$) of SSTE, SUST, GJ-SAMPLE, and DRS, is *inversely* proportional to OUT. Intuitively, if OUT is as large as $\text{AGM}(\mathcal{H})$, a set of samples S_I are highly likely to be the actual join answers of Q . In other words, the join answers are spread over a dense database. A small number of

samples would give us enough information about the distribution of the join answers and estimating OUT accurately. In contrast, if OUT is small, the join answers would be skewed and sparsely distributed in certain regions of a database, and most samples would not be the join answers. Therefore, a small number of samples cannot effectively decrease the uncertainty of estimation, which may require a lot more sampling for an accurate estimation.

5.6 Underlying Index Structure

We explain the underlying index Index_F for each relation R_F to evaluate the operation $\pi_I(R_F \bowtie s)$ in $\tilde{O}(1)$ time, as mentioned in §5.2. We define $F_I = F \cap I$ and $F_s = F \cap \text{Attr}(s)$, where $\text{Attr}(s)$ is the set of attributes of s . Note that F_I and F_s are disjoint in GENERICCARDEST. If $F_s = \{A_1, A_2, \dots, A_m\}$, then $R_F \bowtie s = R_F \bowtie \pi_{F_s}(s) = \sigma_{A_1=\pi_{A_1}s} \sigma_{A_2=\pi_{A_2}s} \dots \sigma_{A_m=\pi_{A_m}s}(R_F)$.

A mild assumption in [31] is to build a B-tree-like index (e.g., a trie index in [1]) under a global attribute order over the whole database. Then, if all attributes in F_s precede all attributes in F_I , $R_F \bowtie s$ is readily available as an index lookup of $\text{Index}_F(s) = \text{Index}_F(\pi_{F_s}(s))$. Furthermore, if no attribute in F lies between F_s and F_I , $\pi_I(R_F \bowtie s)$ is an index lookup $\text{Index}_F(s)$ up to depth $|F_I|$. To exploit these, the selection of I in each step of GENERICCARDEST should be consistent with the global order. Instead, we can build $|F|!$ indexes for R_F , one for each possible attribute order as in [1], to enable arbitrary selection of I .

Remark. The complexity of building an index for R_F is linear, i.e., $\tilde{O}(|R_F|)$ [1]. However, in contrast to the pre-computing overheads in GJ-SAMPLE, the indexing occurs once per database instead of per query. Therefore, its complexity is ignored from the analysis of $T_{\mathcal{H}}$ in §5.5. Due to their simple structures, indexes can be easily extended to dynamic setting where tuples can be inserted and deleted, with a constant or logarithmic update time.

6 ACHIEVING A GENERALIZED BOUND

In this section, we generalize our $\tilde{O}(\frac{\text{AGM}(\mathcal{H})}{\text{OUT}})$ bound for join size estimation using generalized hypertree decompositions (GHDs, see §2) and achieve a better bound than $\tilde{O}(\text{IN}^{f_{\text{htw}}(\mathcal{H})})$, which is the bound of an exact aggregation algorithm AGGROGHDJOIN using GHDs (see §A.3). We leave uniform sampling using GHDs as a future work in §8. Our new bound may be better than our previous bound $\tilde{O}(\frac{\text{AGM}(\mathcal{H})}{\text{OUT}})$, especially when OUT is small (see Example 3). We first provide background about GHDs and related algorithms.

6.1 Aggregations over GHDs

Instead of performing joins, OUT can be more efficiently computed by solving an AJAR (Aggregations and Joins over Annotated Relations) query [25]. AJAR queries assume that each relation R_F is annotated; $R_F = \{(r, \lambda(r))\}$ where each tuple r has an *annotation* $\lambda(r)$ (from some domain $\mathbb{K}$). If two tuples r_1 and r_2 are joined, the joined tuple $r_1 \bowtie r_2$ has $\lambda(r_1) \otimes \lambda(r_2)$ as its annotation. Hence, joining annotated relations results in another annotated relation:

$$\bowtie_{F \in \mathcal{E}} R_F = \{(r, \lambda) \mid r \text{ is a join answer}, \lambda = \otimes_{F \in \mathcal{E}} \lambda(\pi_F r)\}. \quad (4)$$

Furthermore, we can define an aggregation over an annotated relation R with attributes $\mathcal{A}$ by a pair of attribute A and sum operator $\oplus$ [25]. Let $G = \mathcal{A} \setminus \{A\}$. Then, the $(A, \oplus)$ -aggregation of R

generates an annotated relation R_G of attributes G , where G is the *grouping/output attributes*, and λ_G is an aggregated result for r_G :

$$\sum_{A, \oplus} R = \{(r_G, \lambda_G) : r_G \in \pi_G R \text{ and } \lambda_G = \bigoplus_{(r, \lambda) \in R: \pi_G r = r_G} \lambda\}. \quad (5)$$

By definition, no tuple r_G is duplicated in the aggregation result. If multiple aggregation attributes $M = \{A_1, A_2, \dots, A_n\}$ share the same $\oplus$ operator, we simply write $\sum_{A_1, \oplus} \sum_{A_2, \oplus} \dots \sum_{A_n, \oplus}$ as $\sum_{M, \oplus}$. Here, attributes in M are marginalized out and removed from the resulting relation. The output attributes G becomes $\mathcal{A} \setminus M$. Altogether, if $(\mathbb{K}, \oplus, \otimes)$ forms a commutative semiring [21], we can aggregate over joins (of annotated relations) [25] as $\sum_{M, \oplus} \bowtie_{F \in \mathcal{E}} R_F$. If $\mathbb{K} = \mathbb{Z}$, $\lambda(r) = 1 \forall r \in R_F$, $\oplus = +$, $\otimes = \times$, and $M = \mathcal{V}$, the query $\sum_{\mathcal{V}, +} \bowtie_{F \in \mathcal{E}} R_F$ generates an annotated relation with a single tuple r without any attribute. Here, $\lambda(r) = \text{OUT} = |\bowtie_{F \in \mathcal{E}} R_F|$. We hereafter omit $\oplus$ whenever possible without ambiguity. In addition, AJAR queries are known to be more general than FAQ queries [2], for a query can have multiple aggregation operators [25].

Joglekar et al. [25] present AGGROGHDJOIN and AGGROYANNAKAKIS to solve the AJAR queries in $\tilde{O}(\text{IN}^{w^*} + \text{OUT}_{agg})$ time, where w^* is their width of algorithms, and OUT_{agg} is the output size of the aggregation results which is typically smaller than OUT, e.g., 1 in our case. We prove in §A.3 that $w^* \geq fhtw$ for general AJAR queries and $w^* = fhtw$ for computing OUT. Since we show in §6.3 that our new bound using GHDs is smaller than $\tilde{O}(\text{IN}^{fhtw})$, it is also smaller than $\tilde{O}(\text{IN}^{w^*})$.

6.2 Sampling over GHDs

From now, we use the same setting from §6.1 to compute the AJAR query $\sum_{\mathcal{V}} \bowtie_{F \in \mathcal{E}} R_F$. For a set of grouping attributes G , GROUPBYCARDEST (Algorithm 2) returns an approximate answer to the AJAR query $\mathcal{H}^G := \sum_{\mathcal{V} \setminus G} \mathcal{H} = \sum_{\mathcal{V} \setminus G} \bowtie_{F \in \mathcal{E}} R_F$, i.e., an annotated relation $\{(g, Z[g]) | (g, \lambda(g)) \in \mathcal{H}^G\}$ (Lines 2-5); $\lambda(g)$ and $Z[g]$ represent the exact and approximate value of $\text{OUT}(\mathcal{H}_g)$, where $\mathcal{H}_g$ is the residual hypergraph of $\mathcal{H}$ given g (see §5.1). Therefore, $\sum_{(g, \lambda(g)) \in \mathcal{H}^G} \lambda(g) = \text{OUT}(\mathcal{H})$.

Algorithm 2. GROUPBYCARDEST ($\mathcal{H}(\mathcal{V}, \mathcal{E}), G$)

Input: Hypergraph $\mathcal{H}$ and a set of attributes $G \subseteq \mathcal{V}$

- 1: $R \leftarrow \emptyset$
- /* compute $\mathcal{H}^G := \sum_{\mathcal{V} \setminus G} \mathcal{H}$, skip any duplicated g */
- 2: **foreach** ($g \in \text{GENERICJOIN}(\mathcal{H}, G, \emptyset)$) **do**
- 3: $Z[g] \leftarrow \text{GENERICCARDEST}(\mathcal{H}, \mathcal{V} \setminus G, g)$ /* approximation of the output size of the residual query given g */
- 4: $R \leftarrow R \cup \{(g, Z[g])\}$
- 5: **return** R

GHDCARDEST (Algorithm 3) is our entry function given a GHD $(\mathcal{T}, \chi)$. For each node $t \in \mathcal{V}_{\mathcal{T}}$, we define $\mathcal{H}_t := (\chi(t), \mathcal{E}_{\chi(t), \pi})$, i.e., sub-query of $\mathcal{H}$ on t (Line 3), and $G(t)$ as the set of grouping attributes of t , determined as the shared attributes across the nodes (Line 4). GROUPBYCARDEST at Line 5 takes this $G(t)$ as the grouping attributes G and returns an annotated relation $R_t := \{(g_t, Z[g_t]) | (g_t, \lambda(g_t)) \in \mathcal{H}_t^{G(t)}\}$, where $\mathcal{H}_t^{G(t)} = \sum_{\chi(t) \setminus G(t)} \mathcal{H}_t$. Therefore, R_t is an approximation of the aggregation of $\mathcal{H}_t$ with

output attributes $G(t)$. Finally, SIMPLEAGGROYANNAKAKIS at Line 7 takes these R_t 's and runs a simple version of AGGROYANNAKAKIS (see Algorithm 5 in §A.2) to compute $\sum_{G(\mathcal{T})} \bowtie_{t \in \mathcal{V}_{\mathcal{T}}} R_t$ for $G(\mathcal{T}) := \cup_{t \in \mathcal{V}_{\mathcal{T}}} G(t)$, the union of grouping attributes of all GHD nodes. $G(\mathcal{T})$ is used as join attributes between R_t 's in Line 7. Finally, Lemma 3 stats the unbiasedness of GHDCARDEST.

Algorithm 3. GHDCARDEST($\mathcal{H}(\mathcal{V}, \mathcal{E}), (\mathcal{T}(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}}), \chi)$)

Input: Query hypergraph $\mathcal{H}$ and a GHD $(\mathcal{T}, \chi)$ of $\mathcal{H}$

- 1: $S_{\mathcal{T}} \leftarrow \emptyset$
- 2: **foreach** ($t \in \mathcal{V}_{\mathcal{T}}$) **do**
- 3: $\mathcal{H}_t \leftarrow (\chi(t), \mathcal{E}_{\chi(t), \pi})$
- 4: $G(t) \leftarrow \{v \in \chi(t) | \exists t' \in \mathcal{V}_{\mathcal{T}} \text{ s.t., } t' \neq t, v \in \chi(t')\}$
- 5: $R_t \leftarrow \text{GROUPBYCARDEST}(\mathcal{H}_t, G(t))$ /* approximate annotated relation of $\mathcal{H}_t^{G(t)} := \sum_{\chi(t) \setminus G(t)} \mathcal{H}_t$ */
- 6: $S_{\mathcal{T}} \leftarrow S_{\mathcal{T}} \cup R_t$
- 7: **return** SIMPLEAGGROYANNAKAKIS($(\mathcal{T}, \chi), S_{\mathcal{T}}$)

Lemma 3. GHDCARDEST is an unbiased estimator of $|\bowtie_{F \in \mathcal{E}} R_F|$.

Example 2. We use $\mathcal{H}$ and $\mathcal{T}_2$ in Figure 1 as an example. Here, $G(t_0) = \{X, X'\}$, $G(t_1) = \{X\}$, and $G(t_2) = \{X'\}$ are grouping attributes. (d) shows examples of annotated relations R_t 's, which are the outputs of GROUPBYCARDEST on the three nodes. Since we allow duplicate tuples in any base relation $R_F : F \in \mathcal{E}$, annotations for R_{t_0} can be larger than one, even if $\chi(t_0)$ is covered by a single hyperedge and $\chi(t_0) = G(t_0)$. (e) and (f) show the join and aggregation on R_t 's performed in SIMPLEAGGROYANNAKAKIS. Note that the annotations are multiplied in joins, and added in aggregations. The final result, 8450, is an approximation of OUT.

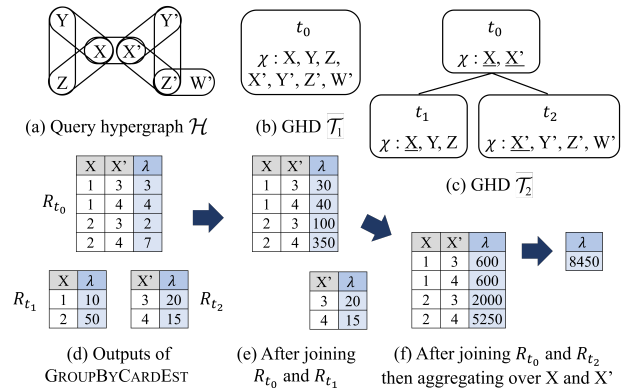


Figure 1: Example of GHDCARDEST using GHD $\mathcal{T}_2$ in (c). Attributes in $G(t)$ are underlined in (c).

The key idea of GHDCARDEST is to pushdown the partial sum $\sum_{\chi(t) \setminus G(t)}$ from $\sum_{\mathcal{V}}$ to each node t in computing R_t , which is similar to the key idea of AGGROGHDJOIN that pushes down the aggregation attributes in GHD as deeply as possible. However, it is slightly different since we shrink each relation obtained from a node *before* the join phase in AGGROYANNAKAKIS; AGGROYANNAKAKIS aggregates *after* joining each pair of parent-child relations in GHD.

We also argue that our definition of $G(t)$ is minimal; if an attribute in $G(t)$ is excluded from R_t , the node t loses a join relationship with another node. Our $G(t)$ also minimizes the runtime of

a non-sampling procedure, GENERICJOIN at Line 2 of Algorithm 2. Its runtime $\tilde{O}(AGM(\mathcal{H}_t^{G(t)}))$ increases with the output attributes $G(t)$ due to the *node-monotone* property of the AGM bound [25].

6.3 Analysis of GHDCARDEST

Using the analyses in §5.5 as building blocks, we analyze the variance and runtime of GHDCARDEST. For each $(g_t, Z[g_t]) \in R_t$ for a GHD node t , we attach two additional annotations $\text{Var}[Z[g_t]]$ and $T[g_t]$ (the runtime of obtaining $Z[g_t]$), and let $\mathcal{H}_{t,g_t}$ (or simply $\mathcal{H}_{g_t}$) denote the residual hypergraph of $\mathcal{H}_t$ given g_t in Line 3 of Algorithm 2. Then, $Z[g_t]$ is an unbiased estimate of $\lambda(g_t) = \text{OUT}(\mathcal{H}_{g_t})$ from Lemma 1. We also define $(f, Z[f])$ as an annotated tuple in $\mathfrak{A}_{t \in \mathcal{V}_T} R_t$ so $Z[f] = \prod_{t \in \mathcal{V}_T} Z[\pi_{G(t)} f]$.

Recall that our primary goal for join size estimation is to perform $(1 \pm \epsilon)$ -approximation of OUT. To use Chebyshev's inequality, we should bound $\text{Var}[Z]$ by $\epsilon^2 \delta \text{OUT}^2$ as in §5.5. Then the following questions arise: 1) Can we make this inequality hold for any ϵ and δ ? 2) How would the runtime T be expressed? To answer these questions, we first express Z using $Z[f]$ values.

$$Z = \sum_{(f, Z[f]) \in \mathfrak{A}_{t \in \mathcal{V}_T} R_t} Z[f]. \quad (6)$$

We arbitrarily order $\mathcal{V}_T$ and regard each f as an ordered set $\{\pi_{G(t)} f | t \in \mathcal{V}_T\}$. Since $Z[g_t]$ values for $g_t \in f$ are mutually independent, we have

$$\mathbb{E}[Z[f]] = \mathbb{E}\left[\prod_{g_t \in f} Z[g_t]\right] = \prod_{g_t \in f} \mathbb{E}[Z[g_t]], \quad (7)$$

$$\begin{aligned} \text{Var}[Z[f]] &= \text{Var}\left[\prod_{g_t \in f} Z[g_t]\right] \\ &= \prod_{g_t \in f} (\text{Var}[Z[g_t]] + \mathbb{E}[Z[g_t]]^2) - \prod_{g_t \in f} \mathbb{E}[Z[g_t]]^2. \end{aligned} \quad (8)$$

While $\mathbb{E}[Z[f]]$ can be simply decomposed into $\prod_{g_t \in f} \mathbb{E}[Z[g_t]]$, $\text{Var}[Z[f]]$ has the internal term $\mathbb{E}[Z[g_t]]^2$ which prevents a simple decomposition. For any two different tuples $f_1, f_2 \in \mathfrak{A}_{t \in \mathcal{V}_T} R_t$, $Z[f_1]$ and $Z[f_2]$ are not independent, since they can have the same sub-tuple, i.e., $g_t \in f_1$ and $g_t \in f_2$ for some $t \in \mathcal{V}_T$. Therefore, we have to consider the covariance $\text{Cov}(Z[f_1], Z[f_2])$ when expanding $\text{Var}[Z]$ below.

$$\text{Var}[Z] = \sum_f \text{Var}[Z[f]] + \sum_{f_1 \neq f_2} \text{Cov}(Z[f_1], Z[f_2]) \quad (9)$$

From the analysis in §5.5, we can arbitrarily control $\text{Var}[Z[g_t]]$ and $T[g_t]$ with the sample size, under a condition that $\text{Var}[Z[g_t]] \cdot T[g_t] = \tilde{O}(AGM(\mathcal{H}_{g_t}) \cdot \text{OUT}(\mathcal{H}_{g_t}))$. In particular, we set $\text{Var}[Z[g_t]] = \tilde{O}(\text{OUT}(\mathcal{H}_{g_t})^2)$ and $T[g_t] = \tilde{O}(\frac{AGM(\mathcal{H}_{g_t})}{\text{OUT}(\mathcal{H}_{g_t})})$.

Lemma 4. $\text{Var}[Z[f]] = \tilde{O}(\mathbb{E}[Z[f]]^2)$ if $\text{Var}[Z[g_t]] = \tilde{O}(\mathbb{E}[Z[g_t]]^2) = \tilde{O}(\text{OUT}(\mathcal{H}_{g_t})^2)$ for every $g_t \in f$.

Lemma 5. $\text{Cov}(Z[f_1], Z[f_2]) = \tilde{O}(\prod_{g_t \in f_1} \mathbb{E}[Z[g_t]] \cdot \prod_{g_t \in f_2} \mathbb{E}[Z[g_t]])$ if the same condition of Lemma 4 holds for f_1 and f_2 .

Now we are ready to answer our first question. By applying Lemmas 4 and 5 to (9), we have

$$\begin{aligned} \text{Var}[Z] &= \sum_f \tilde{O}\left(\prod_{g_t \in f} \mathbb{E}[Z[g_t]]^2\right) \\ &\quad + \sum_{f_1 \neq f_2} \tilde{O}\left(\prod_{g_t \in f_1} \mathbb{E}[Z[g_t]] \prod_{g_t \in f_2} \mathbb{E}[Z[g_t]]\right) \\ &= \tilde{O}\left(\sum_f \prod_{g_t \in f} \mathbb{E}[Z[g_t]]^2 + \sum_{f_1 \neq f_2} \prod_{g_t \in f_1} \mathbb{E}[Z[g_t]] \prod_{g_t \in f_2} \mathbb{E}[Z[g_t]]\right) \\ &= \tilde{O}\left(\left(\sum_f \prod_{g_t \in f} \mathbb{E}[Z[g_t]]\right)^2\right) = \tilde{O}(\text{OUT}^2). \end{aligned} \quad (10)$$

The last equality holds from $\sum_f \prod_{g_t \in f} \mathbb{E}[Z[g_t]] = \sum_f \mathbb{E}[Z[f]] = \mathbb{E}[Z] = \text{OUT}$. As a result, $\text{Var}[Z] = \tilde{O}(\text{OUT}^2)$.

We can make $\text{Var}[Z]$ arbitrarily small, even less than the $\epsilon^2 \delta \text{OUT}^2$ we desire, by setting the constant factor of $\text{Var}[Z[g_t]] = \tilde{O}(\mathbb{E}[Z[g_t]]^2)$ arbitrary small for every g_t . We omit setting the constants which is trivial.

Proposition 9. If $\text{Var}[Z[g_t]]$ approaches to 0 for every g_t , $\text{Var}[Z]$ approaches to 0.

We next answer our second question. GROUPBYCARDEST for each node t takes $\tilde{O}(AGM(\mathcal{H}_t^{G(t)}))$ time at Line 2 and $\sum_{g_t \in R_t} T[g_t]$ at Line 3 of Algorithm 2. SIMPLEAGGROYANNAKAKIS at Line 7 of Algorithm 3 takes $\tilde{O}(\max_{t \in \mathcal{V}_T} AGM(\mathcal{H}_t^{G(t)}))$ time. Therefore, T is $\tilde{O}\left(\max_{t \in \mathcal{V}_T} AGM(\mathcal{H}_t^{G(t)}) + \sum_{t \in \mathcal{V}_T} \sum_{(g_t, Z[g_t]) \in R_t} T[g_t]\right)$. By setting $T[g_t] = \tilde{O}\left(\frac{AGM(\mathcal{H}_{g_t})}{\text{OUT}(\mathcal{H}_{g_t})}\right)$, T becomes $\tilde{O}\left(\max_t AGM(\mathcal{H}_t^{G(t)}) + \sum_t \sum_{(g_t, Z[g_t]) \in R_t} \frac{AGM(\mathcal{H}_{g_t})}{\text{OUT}(\mathcal{H}_{g_t})}\right)$.

We now prove that our new bound is smaller than $\tilde{O}(\text{Infhtw}(\mathcal{H})) = \tilde{O}(\max_t AGM(\mathcal{H}_t))$; $AGM(\mathcal{H}_t^{G(t)}) \leq AGM(\mathcal{H}_t)$ since AGM is node-monotone, and $\sum_t \sum_{g_t} \frac{AGM(\mathcal{H}_{g_t})}{\text{OUT}(\mathcal{H}_{g_t})} \leq \sum_t \sum_{g_t} AGM(\mathcal{H}_{g_t}) \leq AGM(\mathcal{H}_t)$ from Lemma 6 in §B. Therefore, if OUT is $\tilde{O}(\text{Infhtw})$, Infhtw is asymptotically smaller than $\frac{AGM}{\text{OUT}}$ ($AGM = \text{Infhtw}$), proving Theorem 2. We recommend using GENERICCARDEST if OUT is expected to be large, and using GHDCARDEST if OUT is expected to be small.

Example 3. We use the $\mathcal{H}$ in Figure 1 without an edge $\{Z', W'\}$ and assume that every base relation has size N . Then, our previous bound $\tilde{O}(\frac{AGM(\mathcal{H})}{\text{OUT}})$ becomes $\tilde{O}(\frac{N^3}{\text{OUT}})$, from an optimal fractional edge cover x : $x_F = 0$ for $F = \{X, X'\}$ and 0.5 otherwise. Since our new bound is smaller than $\tilde{O}(\text{Infhtw}(\mathcal{H})) = N^{1.5}$, it is also smaller than $\tilde{O}(\frac{N^3}{\text{OUT}})$ if OUT is asymptotically smaller than $N^{1.5}$.

7 OPTIMIZATIONS

This section explains two optimization techniques to enhance estimation efficiency or accuracy.

7.1 Increasing Sampling Probabilities

If we focus on the join size estimation without persisting uniform sampling, we can further reduce the variance. Since $\text{Var}[Z_{\mathcal{H}}] = \sum_{t \in Q} \frac{1}{P(t)} - \text{OUT}^2$ [11, 27] for our method in §5.4, increasing $P(t)$:

$t \in Q$ reduces the variance. First, when m edges tie and have the same maximum $rdeg$ in §5.4, we do not decrease the keeping probability p by m times. This increases the final $P(t)$ by m times. In fact, we can use any sampled F^* from $\mathcal{E}_I$; $P(s_I)$ increases from $\frac{1}{|\mathcal{E}_I|} \cdot \max_{F' \in \mathcal{E}_I} rdeg_{F',s}(s_I) \cdot \frac{AGM(\mathcal{H}_{s \cup s_I})}{\max_{F' \in \mathcal{E}_I} rdeg_{F',s}(s_I) AGM(\mathcal{H}_s)}$ to $\sum_{F \in \mathcal{E}_I} \frac{1}{|\mathcal{E}_I|} \cdot rdeg_{F,s}(s_I) \cdot \frac{AGM(\mathcal{H}_{s \cup s_I})}{\max_{F' \in \mathcal{E}_I} rdeg_{F',s}(s_I) AGM(\mathcal{H}_s)}$. Second, we can interleave GJ-SAMPLE to remove the $\frac{1}{|\mathcal{E}_I|}$ factor in $P(s_I)$. If the $|\Omega_I|$ of GJ-SAMPLE is small enough as a constant, e.g., $\prod_I |\mathcal{E}|$, we can compute $P(s_I) = \frac{AGM(\mathcal{H}_{s \cup s_I})}{AGM(\mathcal{H}_s)}$ for every $s_I \in \Omega_I$ as GJ-SAMPLE.

7.2 Skipping Attributes and GHD Nodes

In GENERICCARDEST, sampling s_I for any non-join attribute is unnecessary and thus, can be safely skipped. For our example query in Figure 1, W' is the only non-join attribute. Let the current sample $s = \{x', y', z'\}$ (attributes are X', Y', Z') and $I = \{W'\}$. Then, sampling any $s_I \in \Omega_I = \pi_{W'}(R_{\{Z', W'\}} \bowtie z')$ does not affect the sample space and sample size for the remaining vertices, i.e., X, Y , and Z . Hence, we skip W' and just return $\frac{1}{P(s_{\{W'\}})} = |\Omega_{\{W'\}}| = |\pi_{W'}(R_{\{Z', W'\}} \bowtie z')|$ at Line 2 instead of 1. If there are multiple non-join attributes $X_1, X_2, \dots, X_n$, we skip all of these, sample for all join attributes only, then return $\prod_{1 \leq i \leq n} |\Omega_{\{X_i\}}|$ at Line 2.

In GHDCARDEST, calling GROUPBYCARDEST for single-edge GHD nodes can be safely skipped. If a GHD node t is covered by a single edge, i.e., $\exists F \in \mathcal{E} : \chi(t) \subseteq F$, we can directly obtain $\mathbb{E}[Z[g_t]] = |R_F \bowtie g_t|$ for every g_t in R_t without sampling in GROUPBYCARDEST. To be consistent, we regard R_t as $\pi_{\chi(t)} R_F$ (with annotations) in order to remove any duplicates in R_t as mentioned in §6.1. t_0 of $\mathcal{T}_2$ in Figure 1 is an example of a single-edge node, where $Z[g_{t_0}] = |R_{\{X, X'\}} \bowtie g_{t_0}|$ for $g_{t_0} \in R_{t_0}$. Therefore, $\text{Var}[Z[g_{t_0}]] = 0$ and $T[g_{t_0}] = \tilde{O}(1)$, reducing both $\text{Var}[Z]$ and T .

8 RESEARCH OPPORTUNITIES

We present six research opportunities based on our study. First, we could express our bound $\tilde{O}(\max_t AGM(\mathcal{H}_t^{G(t)}) + \sum_t \sum_{(g_t, Z[g_t]) \in R_t} \frac{AGM(\mathcal{H}_{g_t})}{OUT(\mathcal{H}_{g_t})})$ in §6 in a more succinct form by removing internal g_t terms, e.g., to $\tilde{O}(\frac{\text{IN}^{fhtw}(\mathcal{H})}{OUT})$. Then, we could more clearly characterize the gap between this bound and $\tilde{O}(\frac{AGM(\mathcal{H})}{OUT})$ or $\tilde{O}(\text{IN}^{fhtw}(\mathcal{H}))$.

Second, we could dynamically change the sampling approach based on the guess of OUT. Our bound $\tilde{O}(\frac{AGM(\mathcal{H})}{OUT})$ in §5 can be small as a constant if OUT is large as $AGM(\mathcal{H})$, while the bound in §6 has advantages over small OUT (e.g., Example 3). As stated in §5.5, we could start with GENERICCARDEST assuming a large OUT, increase the sample size, adjust the assumption, and change to GHDCARDEST if the estimated OUT becomes small.

Third, we could even change between sampling and join, then apply the same analysis. While the join algorithms have zero error but +OUT term in their runtime, sampling algorithms have non-zero errors but $\frac{1}{OUT}$ factor in their runtime. Therefore, it might be possible to balance between sampling and join to reduce the runtime and error for both large- and small-OUT cases, i.e., select a sampling-like algorithm for large OUT and a join-like algorithm for small OUT.

Fourth, we could develop join size estimators with lower complexities using more information about relations, e.g., degree constraints or functional dependencies [3] other than the cardinality constraints in Definition 1.

Fifth, we could extend GHDCARDEST to perform uniform sampling over arbitrary GHDs from the observations in §A.2 that 1) EXACTWEIGHT [34] performs uniform sampling over certain types of GHDs for acyclic joins, and 2) GHDCARDEST can be easily modified for uniform sampling over the same GHDs. In Algorithm 6, the initial value of $W(g_t)$ (sampling weight for a tuple g_t in R_t) is set to $\mathbb{E}[Z[g_t]] = |R_{F_t} \bowtie g_t|$. Therefore, we could extend EXACTWEIGHT and GHDCARDEST for arbitrary GHDs for even cyclic joins, starting from initializing $W(g_t)$ with an estimate $Z[g_t]$ instead of $\mathbb{E}[Z[g_t]]$.

Sixth, we could extend our algorithms to more general problems, approximate counting and uniform sampling for conjunctive queries [5, 16] or join-project queries $\pi_O(Q)$, where O is the set of output attributes [11]. We explain in §C that our degree-based rejection sampling can be easily extended for join-project queries, following the same analysis in Section 3 of [11]. We achieve $\tilde{O}(\frac{AGM(Q)}{|\pi_O(Q)|})$ runtime which is again smaller than that of GJ-SAMPLE.

9 CONCLUSION

In this paper, we have presented a new sampling-based method for join size estimation and uniform sampling. Our method solves an open problem in the literature [11], achieving $(1 \pm \epsilon)$ -approximation of OUT in $\tilde{O}(\frac{AGM}{OUT})$ time for arbitrary join queries. We presented a unified approach that explains and analyzes four known methods: SSTE, SUST, ALLEY, and GJ-SAMPLE. We then extended our method using GHDs to achieve a better bound than $\tilde{O}(\frac{AGM}{OUT})$ for small OUT and optimized the efficiency and accuracy using two approaches. Finally, we have highlighted several interesting research opportunities building on our results. We believe that our work may facilitate studies on achieving lower bounds on the uniform sampling and size estimation over joins.

10 ACKNOWLEDGEMENT

We thank Yufei Tao for pointing out the condition of Lemma 2.

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2021-0-00859, Development of a distributed graph DBMS for intelligent processing of big graphs, 34%), Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2018-0-01398, Development of a Conversational, Self-tuning DBMS, 33%), and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No.NRF-2021R1A2B5B03001551, 33%).

REFERENCES

- [1] Christopher R Aberger, Andrew Lamb, Susan Tu, Andres Nötzli, Kunle Olukotun, and Christopher Ré. 2017. Emptyheaded: A relational engine for graph processing. *ACM Transactions on Database Systems (TODS)* 42, 4 (2017), 1–44.
- [2] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. 2016. FAQ: Questions Asked Frequently. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (San Francisco, California, USA) (PODS '16). Association for Computing Machinery, New York, NY, USA, 13–28. <https://doi.org/10.1145/2902251.2902280>
- [3] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog have to do with

- one another?. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 429–444.
- [4] Maryam Aliakbarpour, Amartya Shankha Biswas, Themis Gouleakis, John Peebles, Ronitt Rubinfeld, and Anak Yodpinyanee. 2018. Sublinear-time algorithms for counting star subgraphs via edge sampling. *Algorithmica* 80, 2 (2018), 668–697.
- [5] Marcelo Arenas, Luis Alberto Croquevielle, Rajesh Jayaram, and Cristian Riveros. 2021. When is approximate counting for conjunctive queries tractable?. In *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing*. 1015–1027.
- [6] Sepehr Assadi, Michael Kapralov, and Sanjeev Khanna. 2019. A simple sublinear-time algorithm for counting arbitrary subgraphs via edge sampling. In *10th Innovations in Theoretical Computer Science, ITCS 2019*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik GmbH, Dagstuhl Publishing, 6.
- [7] Albert Atserias, Martin Grohe, and Daniel Marx. 2013. Size bounds and query plans for relational joins. *SIAM J. Comput.* 42, 4 (2013), 1737–1767.
- [8] Suman K Bera and Amit Chakrabarti. 2017. Towards tighter space bounds for counting triangles and other substructures in graph streams. In *34th Symposium on Theoretical Aspects of Computer Science*.
- [9] Johann Brault-Baron. 2016. Hypergraph acyclicity revisited. *ACM Computing Surveys (CSUR)* 49, 3 (2016), 1–26.
- [10] Nofar Carmeli, Shai Zeevi, Christoph Berkholz, Benny Kimelfeld, and Nicole Schweikardt. 2020. Answering (unions of) conjunctive queries using random access and random-order enumeration. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 393–409.
- [11] Yu Chen and Ke Yi. 2020. Random Sampling and Size Estimation Over Cyclic Joins. In *23rd International Conference on Database Theory (ICDT 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik.
- [12] Shiyuan Deng, Shangqi Lu, and Yufei Tao. 2023. On Join Sampling and Hardness of Combinatorial Output-Sensitive Join Algorithms. In *Proceedings of the 42nd ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2023)*.
- [13] Talya Eden, Amit Levi, Dana Ron, and C Seshadhri. 2017. Approximately counting triangles in sublinear time. *SIAM J. Comput.* 46, 5 (2017), 1603–1646.
- [14] Talya Eden, Dana Ron, and C Seshadhri. 2020. On approximating the number of k-cliques in sublinear time. *SIAM J. Comput.* 49, 4 (2020), 747–771.
- [15] Hendrik Fichtenberger, Mingze Gao, and Pan Peng. 2020. Sampling Arbitrary Subgraphs Exactly Uniformly in Sublinear Time. In *ICALP*.
- [16] Jacob Focke, Leslie Ann Goldberg, Marc Roth, and Stanislav Zivný. 2022. Approximately counting answers to conjunctive queries with disequalities and negations. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 315–324.
- [17] Ehud Friedgut and Jeff Kahn. 1998. On the number of copies of one hypergraph in another. *Israel Journal of Mathematics* 105, 1 (1998), 251–256.
- [18] Oded Goldreich. 2017. *Introduction to property testing*. Cambridge University Press.
- [19] Georg Gottlob, Gianluigi Greco, Nicola Leone, and Francesco Scarcello. 2016. Hypertree decompositions: Questions and answers. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 57–74.
- [20] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2002. Hypertree decompositions and tractable queries. *J. Comput. System Sci.* 64, 3 (2002), 579–627.
- [21] Todd J Green, Grigoris Karvounarakis, and Val Tannen. 2007. Provenance semirings. In *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 31–40.
- [22] Martin Grohe and Daniel Marx. 2014. Constraint solving via fractional edge covers. *ACM Transactions on Algorithms (TALG)* 11, 1 (2014), 1–20.
- [23] Daniel G Horvitz and Donovan J Thompson. 1952. A generalization of sampling without replacement from a finite universe. *Journal of the American Statistical Association* 47, 260 (1952), 663–685.
- [24] Mark R Jerrum, Leslie G Valiant, and Vijay V Vazirani. 1986. Random generation of combinatorial structures from a uniform distribution. *Theoretical computer science* 43 (1986), 169–188.
- [25] Manas R. Joglekar, Rohan Puttagunta, and Christopher Ré. 2016. AJAR: Aggregations and Joins over Annotated Relations. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (San Francisco, California, USA) (PODS '16). Association for Computing Machinery, New York, NY, USA, 91–106. <https://doi.org/10.1145/2902251.2902293>
- [26] Kyoungmin Kim, Jaehyun Ha, George Fletcher, and Wook-Shin Han. 2023. full paper. *arXiv preprint arXiv:2304.00715* (2023).
- [27] Kyoungmin Kim, Hyeonji Kim, George Fletcher, and Wook-Shin Han. 2021. Combining Sampling and Synopses with Worst-Case Optimal Runtime and Quality Guarantees for Graph Pattern Cardinality Estimation. In *Proceedings of the 2021 International Conference on Management of Data*. 964–976.
- [28] Viktor Leis, Andrey Gubichev, Athanasios Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2015. How good are query optimizers, really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215.
- [29] Feifei Li, Bin Wu, Ke Yi, and Zhuoyue Zhao. 2016. Wander join: Online aggregation via random walks. In *Proceedings of the 2016 International Conference on*

Management of Data. 615–629.

- [30] Hung Q Ngo, Ely Porat, Christopher Ré, and Atri Rudra. 2012. Worst-case Optimal Join Algorithms. (2012).
- [31] Hung Q Ngo, Christopher Ré, and Atri Rudra. 2014. Skew strikes back: New developments in the theory of join algorithms. *ACM SIGMOD Record* 42, 4 (2014), 5–16.
- [32] Todd L Veldhuizen. 2012. Leapfrog trijoin: a worst-case optimal join algorithm. *arXiv preprint arXiv:1210.0481* (2012).
- [33] Mihalis Yannakakis. 1981. Algorithms for acyclic database schemes. In *VLDB*, Vol. 81. 82–94.
- [34] Zhuoyue Zhao, Robert Christensen, Feifei Li, Xiao Hu, and Ke Yi. 2018. Random sampling over joins revisited. In *Proceedings of the 2018 International Conference on Management of Data*. 1525–1539.

APPENDIX

A ALGORITHMS

A.1 GENERICJOIN

GENERICJOIN (Algorithm 4) [31] is a representative worst-case optimal join algorithm. From the set of output attributes O (initially $\mathcal{V}$), it selects a subset I where $1 \leq |I| < |O|$ (Line 4). For each join answer s_I of an induced hypergraph of $\mathcal{H}$ projected onto I (Line 5), it bounds attributes I to s_I and proceeds to match the residual hypergraph (Line 6). If $|O| = 1$, the results are obtained from the intersection ($\cap$) operation (Lines 1-2). The runtime of GENERICJOIN is $\tilde{O}(AGM(\mathcal{H}))$, which can be proven from Lemma 6 [31].

Algorithm 4. GENERICJOIN($\mathcal{H}(\mathcal{V}, \mathcal{E})$, O , s)

Input: Query hypergraph $\mathcal{H}$, output attributes $O \subseteq \mathcal{V}$, and current tuple s

```

1: if ( $|O| = 1$ ) then
2:   return  $\cap_{F \in \mathcal{E}_O} \pi_O(R_F \bowtie s)$ 
3:  $R \leftarrow \emptyset$ 
4:  $I \leftarrow$  a non-empty proper subset of  $O$ 
5: for each ( $s_I \in \text{GENERICJOIN}(\mathcal{H}, I, s)$ ) do
6:    $R[s_I] \leftarrow \text{GENERICJOIN}(\mathcal{H}, O \setminus I, s \upharpoonright_{s_I})$ 
7:    $R \leftarrow R \cup \{s_I\} \times R[s_I]$ 
8: return  $R$ 
```

A.2 YANNAKAKIS, SIMPLEAGGROYANNAKAKIS, and EXACTWEIGHT

In this section and in Algorithms 5–6, we assume that the root node r of any GHD $\mathcal{T}$ has a virtual parent node pr where 1) $pr \notin \mathcal{V}_{\mathcal{T}}$, 2) $\chi(pr) = \emptyset$, and 3) R_{pr} (input relation for pr) contains a single tuple g_{pr} that joins with all tuples in R_r , for ease of explanation. As mentioned in §6.1, we assume no duplicate tuples in each R_t .

Given a join tree $\mathcal{T}$ of a α -acyclic query Q [9], YANNAKAKIS [33] performs the join in $\tilde{O}(\text{IN} + \text{OUT})$ time using dynamic programming. Algorithm 5 without Lines 8–9 and setting $R' = R_t$ is YANNAKAKIS, where the bottom-up and top-down semi-join reductions (Lines 1–6) are followed by the bottom-up join (Lines 7–11). Since the semi-join reductions remove dangling tuples (that do not participate in the final join answers), the intermediate join size monotonically increases up to OUT in the bottom-up join. Therefore, the total runtime is $\tilde{O}(\text{IN} + \text{OUT})$.

SIMPLEAGGROYANNAKAKIS (Algorithm 5) is a simplified version of AGGROYANNAKAKIS [25], where 1) all attributes are used for aggregation and 2) sum is the only aggregation operator. AGGROYANNAKAKIS can handle a more general class of aggregation queries, e.g., sum and max operators are used for different attributes. In

Algorithm 5. SIMPLEAGGROYANNAKAKIS($(\mathcal{T}(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}}), \chi), \{R_t | t \in \mathcal{V}_{\mathcal{T}}\}$)

Input: GHD $(\mathcal{T}, \chi)$ and relations R_t for each $t \in \mathcal{V}_{\mathcal{T}}$

```

1: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some bottom-up order) do
2:    $p \leftarrow \text{parent of } t$ 
3:    $R_p \leftarrow R_p \bowtie R_t$ 
4: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some top-down order) do
5:    $p \leftarrow \text{parent of } t$ 
6:    $R_t \leftarrow R_t \bowtie R_p$ 
7: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some bottom-up order) do
8:    $\beta \leftarrow \{A \in G(t) | \text{TOP}_{\mathcal{T}}(A) = t\}$ 
9:    $R' \leftarrow \sum_{\beta} R_t$ 
10:   $p \leftarrow \text{parent of } t$ 
11:   $R_p \leftarrow R_p \bowtie R'$ 
12: return  $R_r$  for the root  $r$ 

```

Line 8, $\text{TOP}_{\mathcal{T}}(A)$ denotes the *top* node of an attribute A , which is the closest node to the root among $\{t \in \mathcal{V}_{\mathcal{T}} | \chi(t) \ni A\}$. $G(t)$ is the set of output attributes of R_t . Line 9 aggregates on the attributes in $G(t)$ having t as their top node, since these attributes do not appear in ancestors and are marginalized out. This early marginalization is the key idea of maintaining the runtime of SIMPLEAGGROYANNAKAKIS to be $\tilde{O}(\text{IN} + \text{OUT}_{agg})$ [25], where OUT_{agg} is the output size of aggregation. For example, $\text{OUT}_{agg} = 1$ if all attributes are marginalized out in computing the join size.

Algorithm 6. EXACTWEIGHT($(\mathcal{T}(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}}), \chi), \{R_t | t \in \mathcal{V}_{\mathcal{T}}\}$)

Input: GHD $(\mathcal{T}, \chi)$ where $\exists F_t \in \mathcal{E} : \chi(t) \subseteq F_t$ for every $t \in \mathcal{V}_{\mathcal{T}}$ and relations R_t for each $t \in \mathcal{V}_{\mathcal{T}}$

```

1:  $W(g_t) \leftarrow |R_{F_t} \bowtie g_t|$  for every  $t \in \mathcal{V}_{\mathcal{T}}$  and  $g_t \in R_t$ 
2: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some bottom-up order) do
3:    $p \leftarrow \text{parent of } t$ 
4:   foreach ( $g_p \in R_p$ ) do
5:      $W(g_p, R_t) \leftarrow \sum_{g_t \in R_t \bowtie g_p} W(g_t)$ 
6:      $W(g_p) \leftarrow W(g_p) \cdot W(g_p, R_t)$ 
7:    $R_p \leftarrow R_p \bowtie R_t$ 
8:  $s \leftarrow g_{pr}$ 
9: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some top-down order) do
10:   $p \leftarrow \text{parent of } t$ 
11:   $s_p \leftarrow \pi_{\chi(p)} s$ 
12:   $\{s_t\} \leftarrow \text{SAMPLEFROM}(R_t \bowtie s_p, 1, \{\frac{W(g_t)}{W(s_p, R_t)} | g_t \in R_t \bowtie s_p\})$ 
13:  /* sample a tuple  $s_t$  from  $R_t \bowtie s_p$  with
14:     probability  $\frac{W(s_t)}{W(s_p, R_t)}$  */
15:   $s \leftarrow s \cup s_t$ 
16: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$  in some order) do
17:  Replace  $s_t$  in  $s$  in-place with a uniform sample from  $R_{F_t} \bowtie s_t$ 
18: return  $s$ 

```

We explain EXACTWEIGHT [34] in our context of using GHDs (Algorithm 6), but assume that R_t 's are not annotated and compute each weight W explicitly. EXACTWEIGHT performs uniform sampling over acyclic joins, where the GHDs of single-edge nodes are given. EXACTWEIGHT computes the sampling weights $W(g_p)$ and $W(g_p, R_t)$ for each tuple g_p and a child relation R_t bottom-up (Lines 2-7) and samples tuples proportional to their weights top-down (Lines 8-13). The sample is then replaced with the tuples sampled

from the base relations (Lines 14-15). The bottom-up part is the preprocessing step that takes $\tilde{O}(|\mathcal{V}_{\mathcal{T}}| \sum_{t \in \mathcal{V}_{\mathcal{T}}} |R_t|) = \tilde{O}(\max_t |R_{F_t}|) = \tilde{O}(\text{IN})$ time, and the top-down sampling and replacement takes $\tilde{O}(1)$ for each sample and can be performed multiple times after the preprocessing.

We now briefly explain why EXACTWEIGHT returns a uniform sample. $W(g_p, R_t)$ corresponds to the join size between g_p and the child subtree of p rooted at t , and $W(g_p)$ corresponds to the join size between g_p and all child subtrees of p . Hence, $W(g_{pr}) = \text{OUT}$ which is the join size of the whole tree. For a node p , its children $\{t_1, t_2, \dots, t_m\}$ and its parent pp , $P(s_p) = \frac{W(s_p)}{W(s_{pp}, R_p)}$ and $P(s_{t_i}) = \frac{W(s_{t_i})}{W(s_p, R_{t_i})}$ for $i \in [1, m]$. Therefore, $P(s_p, s_{t_1}, \dots, s_{t_m}) = \frac{W(s_p)}{W(s_{pp}, R_p)} \prod_{1 \leq i \leq m} \frac{W(s_{t_i})}{W(s_p, R_{t_i})} = \frac{|R_{F_p} \bowtie s_p| \prod_i W(s_p, R_{t_i})}{W(s_{pp}, R_p)} \prod_i \frac{W(s_{t_i})}{W(s_p, R_{t_i})} = \frac{|R_{F_p} \bowtie s_p| \prod_i W(s_{t_i})}{W(s_{pp}, R_p)}$. We have seen a similar elimination of probability terms in eq. (3), which leads to $P(s) = \frac{\prod_t |R_{F_t} \bowtie s_t|}{W(g_{pr}, R_r)} = \frac{\prod_t |R_{F_t} \bowtie s_t|}{\text{OUT}}$ after Line 13. By uniformly sampling a tuple from each $R_{F_t} \bowtie s_t$ at Line 15, $P(s)$ becomes $\frac{1}{\text{OUT}}$ at Line 16.

We can also make GHDCARDEST in Algorithm 3 perform uniform sampling for the same GHD by 1) iterating t bottom-up at Line 2, 2) adding the same initialization and updates of sampling weights, and 3) adding the same sampling part. Then, the preprocessing step of GHDCARDEST also takes $\tilde{O}(\text{IN})$ as EXACTWEIGHT. From this perspective, EXACTWEIGHT is similar to a specific instance of GHDCARDEST for GHDs of single-edge nodes and acyclic queries. Extending the above modifications of GHDCARDEST to work for arbitrary GHDs and cyclic queries, would be an interesting future work. We could also extend our work to unions of acyclic conjunctive queries and sampling *without* replacement, as what [10] is to [34].

A.3 GHDJOIN and AGGROGHDJOIN

Given a generalized hypertree decomposition (GHD) of a query, GHDJOIN (Algorithm 7) [25] uses $\mathcal{T}$ as a join tree and performs GENERICJOIN to obtain the join answers of each node (Lines 2-4). $\mathcal{H}_t$ is the subquery defined on node t (Line 3). Then, it performs YANNAKAKIS over these join answers to evaluate the original query (Line 5). Therefore, the runtime of GHDJOIN is $\tilde{O}(\max_{t \in \mathcal{V}_{\mathcal{T}}} \text{IN}^{\rho(\mathcal{H}_t)} + \text{OUT}) = \tilde{O}(\text{IN}^{fhtw(\mathcal{T}, \mathcal{H})} + \text{OUT})$ from Definition 3.

Algorithm 7. GHDJOIN($(\mathcal{H}(\mathcal{V}, \mathcal{E}), (\mathcal{T}(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}}), \chi))$)

Input: Query hypergraph $\mathcal{H}$ and a GHD $(\mathcal{T}, \chi)$ of $\mathcal{H}$

```

1:  $S_{\mathcal{T}} \leftarrow \emptyset$ 
2: foreach ( $t \in \mathcal{V}_{\mathcal{T}}$ ) do
3:    $\mathcal{H}_t \leftarrow (\chi(t), \mathcal{E}_{\chi(t), \pi})$ 
4:   /* compute  $\mathcal{H}_t$  */
5:    $S_{\mathcal{T}} \leftarrow S_{\mathcal{T}} \cup \text{GENERICJOIN}(\mathcal{H}_t, \chi(t), \emptyset)$ 
6: return YANNAKAKIS( $(\mathcal{T}, \chi), S_{\mathcal{T}}$ )

```

AGGROGHDJOIN [25] is an aggregation version of GHDJOIN. It is different from GHDJOIN in two aspects: 1) calling AGGROYANNAKAKIS instead of YANNAKAKIS and 2) including some extra work to ensure that each annotation of a tuple is passed to exactly one

call of **GENERICJOIN** [25]. Note that a hyperedge $F \in \mathcal{E}$ can be visited in multiple nodes, thus, its annotations may be aggregated multiple times, leading to an incorrect output.

From the runtime of **GENERICJOIN** and **AGGROYANNAKAKIS**, the runtime of **AGGROGHDJOIN** is $\tilde{O}(\text{IN}^{fhtw(\mathcal{T}, \mathcal{H})} + \text{OUT}_{agg})$. In [25], the runtime of **AGGROGHDJOIN** is expressed as $\tilde{O}(\text{IN}^{w^*} + \text{OUT}_{agg})$, where $w^* := \min_{(\mathcal{T}, \chi) \in \text{Val}} fhtw(\mathcal{T}, \mathcal{H})$. Here, Val denotes the set of valid GHDs of $\mathcal{H}$, that preserve the aggregation ordering of the specified aggregation $\sum_{A_1, \oplus_1} \sum_{A_2, \oplus_2} \dots \sum_{A_n, \oplus_n}$; changing the order of $\sum_{A_i, \oplus_i}$ and $\sum_{A_j, \oplus_j}$ might vary the aggregation result [25]. Since $fhtw(\mathcal{H}) = \min_{(\mathcal{T}, \chi) \in \text{Val}} fhtw(\mathcal{T}, \mathcal{H})$ from Definition 3 and Val is a subset of the set of all GHDs, $w^* \geq fhtw(\mathcal{H})$. However, as in our case where all aggregation operators are the same ($\oplus_i = +$), thus commutative, any aggregation order gives the same aggregation result. Therefore, Val becomes the set of all GHDs. This leads to $w^* = fhtw(\mathcal{H})$.

B QUERY DECOMPOSITION LEMMAS

This section explains two similar query decomposition lemmas proved by Ngo et al. [31] and Chen & Yi [11].

Lemma 6. [31]: *Given a query hypergraph $\mathcal{H}(\mathcal{V}, \mathcal{E})$, let x be any fractional edge cover of $\mathcal{H}$. Let I be an arbitrary non-empty proper subset of $\mathcal{V}$, $J = \mathcal{V} \setminus I$, and $L = \bowtie_{F \in \mathcal{E}_I} \pi_I R_F$. Then,*

$$\sum_{s_I \in L'} \prod_{F \in \mathcal{E}_J} |R_F \bowtie s \uplus s_I|^{x_F} \leq \prod_{F \in \mathcal{E}} |R_F|^{x_F}. \quad (11)$$

Using this lemma, Ngo et al. [31] prove that **GENERICJOIN** has $\tilde{O}(\prod_{F \in \mathcal{E}} |R_F|^{x_F})$ runtime (i.e., the RHS above) using an induction hypothesis on $|\mathcal{V}|$. If x is an optimal fractional edge cover, the RHS becomes $\text{AGM}(\mathcal{H})$.

We briefly explain this with Algorithm 4. If we replace 1) $\mathcal{V}$ in the lemma with O in Algorithm 4 and 2) R_F with $R_F \bowtie s$, we obtain $L = \bowtie_{F \in \mathcal{E}_I} \pi_I (R_F \bowtie s)$ and

$$\sum_{s_I \in L'} \prod_{F \in \mathcal{E}_{O \setminus I}} |R_F \bowtie s \uplus s_I|^{x_F} \leq \prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}. \quad (12)$$

If $|O| = 1$, Line 2 takes $\tilde{O}(\min_{F \in \mathcal{E}_O} |\pi_O(R_F \bowtie s)|)$ time where $\min_{F \in \mathcal{E}_O} |\pi_O(R_F \bowtie s)| \leq \prod_{F \in \mathcal{E}_O} |\pi_O(R_F \bowtie s)|^{x_F} \leq \prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}$ for $\sum_{F \in \mathcal{E}_O} x_F \geq 1$. The last term is equivalent to the RHS above.

If $|O| > 1$, Lines 5-6 take $\tilde{O}(|L| + \sum_{s_I \in L} \prod_{F \in \mathcal{E}_{O \setminus I}} |\pi_{O \setminus I}(R_F \bowtie s \uplus s_I)|^{x_F})$ time where $|L| \leq \prod_{F \in \mathcal{E}_I} |R_F \bowtie s|^{x_F}$ from the induction hypothesis ($\because |I| < |O|$, and $\prod_{F \in \mathcal{E}_I} |R_F \bowtie s|^{x_F} \leq \prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}$) and $\sum_{s_I \in L} \prod_{F \in \mathcal{E}_{O \setminus I}} |\pi_{O \setminus I}(R_F \bowtie s \uplus s_I)|^{x_F} \leq \sum_{s_I \in L} \prod_{F \in \mathcal{E}_{O \setminus I}} |R_F \bowtie s \uplus s_I|^{x_F} \leq \prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}$ from the lemma, which is again the RHS above.

Therefore, in both cases, the runtime of Algorithm 4 is the big-Oh of the RHS above.

Lemma 7. [11]: *Given a query hypergraph $\mathcal{H}(\mathcal{V}, \mathcal{E})$, let x be any fractional edge cover of $\mathcal{H}$. Let $I = \{A\}$ for an arbitrary attribute $A \in \mathcal{V}$, $J = \mathcal{V} \setminus I$, and $L' = \pi_I R_{F^{GJ}}$ for $F^{GJ} = \arg \min_{F \in \mathcal{E}_I} |\pi_I R_F|$. Then,*

$$\sum_{s_I \in L'} \prod_{F \in \mathcal{E}_J} |R_F \bowtie s_I|^{x_F} \leq \prod_{F \in \mathcal{E}} |R_F|^{x_F}. \quad (13)$$

Since Chen & Yi [11] explain Lemma 7 without a name, we also call it the query decomposition lemma since it is similar to Lemma 6. However, it is different from Lemma 6 in that 1) $|I|$ must be 1 and 2) L' is obtained without an actual join or intersection in **GENERICJOIN**. If $|I| = 1$ in Lemma 6, then $L = \bowtie_{F \in \mathcal{E}_I} \pi_I R_F = \cap_{F \in \mathcal{E}_I} \pi_I R_F$. Since $F^{GJ} \in \mathcal{E}_I$, $L' \supseteq L$. Therefore, Lemma 7 is stronger than Lemma 6 if $|I| = 1$.

Lemma 7 is the key to show that $\sum_{s_I \in \Omega_I} P(s_I) \leq 1$ for **GJ-SAMPLE**. By replacing 1) $\mathcal{V}$ in the above lemma with O in **GJ-SAMPLE** in §5.3 and 2) R_F with $R_F \bowtie s$, we obtain $L' = \pi_I (R_{F^{GJ}} \bowtie s) = \Omega_I$ for $F^{GJ} = \arg \min_{F \in \mathcal{E}_I} |\pi_I (R_F \bowtie s)|$ and

$$\sum_{s_I \in L'} \prod_{F \in \mathcal{E}_{O \setminus I}} |R_F \bowtie s \uplus s_I|^{x_F} \leq \prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}, \quad (14)$$

resulting in

$$\begin{aligned} \sum_{s_I \in \Omega_I} P(s_I) &= \sum_{s_I \in L'} \frac{\text{AGM}(\mathcal{H}_{s \uplus s_I})}{\text{AGM}(\mathcal{H}_s)} \\ &= \sum_{s_I \in L'} \frac{\prod_{F \in \mathcal{E}_{O \setminus I}} |R_F \bowtie s \uplus s_I|^{x_F}}{\prod_{F \in \mathcal{E}_O} |R_F \bowtie s|^{x_F}} \leq 1. \end{aligned} \quad (15)$$

C EXTENSION TO CONJUNCTIVE QUERIES

From the introduction, the join query we have considered so far is a full conjunctive query where all attributes are output attributes. In this section, we easily extend our degree-based rejection sampling to approximate counting and uniform sampling for join-project queries (a.k.a. conjunctive queries) [5, 11, 16], following the same analysis of Section 3 in [11].

Let $\pi_O(Q)$ be a join-project query, where O is the set of projection/output attributes and Q is a join query. We can use the two-step approach in [11]: (Step 1) use Algorithm 1 to sample a join result s from $Q_O := \bowtie_{F \in \mathcal{E}_O} \pi_O(R_F)$, i.e., join the relations that contain any attribute in O only. Here, if Q_O is disconnected, we take a sample from each connected component. (Step 2) check if $Q_{\mathcal{V} \setminus O}(s) := \bowtie_{F \in \mathcal{E}_{\mathcal{V} \setminus O}} (\pi_{\mathcal{V} \setminus O}(R_F \bowtie s))$ is empty. If not, we return s as a sampled result, otherwise, we repeat. Then, the final s returned is a result in $\pi_O(Q)$. Using the same analysis of [11], sampling a tuple in $\pi_O(Q)$ takes $\tilde{O}(\frac{\text{AGM}(Q)}{\text{OUT}(\pi_O(Q))})$ time in expectation for our method.

Step 1 of our algorithm takes time $\tilde{O}(\frac{\text{AGM}(Q_O)}{\text{OUT}(Q_O)})$ in expectation. Step 2 takes time $\tilde{O}(\text{AGM}(Q_{\mathcal{V} \setminus O}(s)))$, using any worst-case optimal join algorithm, e.g., **GENERICJOIN**. Because each $s \in Q_O$ is sampled with probability $\frac{1}{\text{OUT}(Q_O)}$, the expected runtime of Step 2 is (the big-Oh of) $\sum_{s \in Q_O} \frac{\text{AGM}(Q_{\mathcal{V} \setminus O}(s))}{\text{OUT}(Q_O)} \leq \frac{\text{AGM}(Q)}{\text{OUT}(Q_O)}$ where the inequality follows from the query decomposition lemma (Lemma 6). Please refer to Section 3 of [11] for the remaining part of the analysis proving the $\tilde{O}(\frac{\text{AGM}(Q)}{\text{OUT}(\pi_O(Q))})$ time.

Here, we would like to address the complexity of computing $\pi_O(R_F)$ before calling Algorithm 1. If there exists an index for R_F with the attribute order such that the attributes in $O \cap F$ precede the attributes in $F \setminus O$, then $\pi_O(R_F)$ is readily available (i.e., in $\tilde{O}(1)$ time) as explained in §5.6. This results in total $\tilde{O}(\frac{\text{AGM}(Q)}{\text{OUT}(\pi_O(Q))})$ time. Otherwise, we need an additional linear-time preprocessing

to obtain $\pi_O(R_F)$ from R_F , resulting in total $\tilde{O}(\frac{AGM(Q)}{OUT(\pi_O(Q))} + IN)$ time. Nevertheless, we emphasize that this is still lower than the complexity of GJ-SAMPLE for non-sequenceable join-project queries, which is $\tilde{O}(IN \cdot \frac{AGM(Q)}{OUT(\pi_O(Q))} + IN)$ [11].

D COMPONENT-AT-A-TIME SAMPLING

For each component, GENERICCARDEST (Algorithm 1) is called in two recursions, e.g., if $\mathcal{H}$ consists of four odd cycles and three stars, the maximum recursion depth is $2 \cdot (4 + 3) + 1 = 15$, including the calls that reach Line 2. Since the components are vertex-disjoint, i.e., variable-disjoint, $R_F \bowtie s = R_F$ whenever F and (the attributes of) s are contained in different components. Therefore, we can safely drop $\bowtie s$ from the following explanations for the component-at-a-time methods.

We first explain for a star S of n edges $F_1, F_2, \dots, F_n$. Let A be the center attribute, i.e., $\{A\} = \cap_{i \in [1, n]} F_i$. For SSTE, in the first call to GENERICCARDEST, I becomes the (singleton set of) center attribute, i.e., $I = \{A\}$, $\Omega_I = \pi_A R_{F_1}$ and $k = 1$. To sample a vertex a from Ω_I , SSTE first samples an edge e from R_{F_1} , and let $a = \pi_A(e)$. Then, $P(a) = \frac{|R_{F_1} \bowtie a|}{|R_{F_1}|}$ since $|R_{F_1} \bowtie a|$ edges in R_{F_1} have a as their projection onto A . When GENERICCARDEST is called for the second time, I becomes the remaining attributes of S , i.e., $I = \cup_i F_i \setminus \{A\}$, $\Omega_I = \prod_i \pi_I(R_{F_i} \bowtie a)$, and $k = 1$; a sequence of n vertices $(b_1, b_2, \dots, b_n)$ is sampled from Ω_I once (i.e., b_i is sampled from $\pi_I(R_{F_i} \bowtie a)$). If $(a, b_1, b_2, \dots, b_n)$ forms a star in the database, $I[\cdot]$ for these two calls become 1, and the recursion proceeds to the remaining components, with $O \leftarrow O \setminus (\cup_i F_i)$ and $s \leftarrow s \uplus (a, b_1, b_2, \dots, b_n)$.

For SUST, in the first call to GENERICCARDEST, I is simply the set of all attributes of S and $\Omega_I = \prod_i R_{F_i}$, $P(\cdot) = \frac{1}{|\Omega_I|}$, and $k = 1$; a sequence of n edges is sampled once. $I[\cdot]$ is 1 iff they share a common vertex for the center. The second call to GENERICCARDEST is a dummy that directly proceeds to the remaining components.

We next explain for an odd cycle C of $2n + 1$ edges with the vertex (attribute) sequence $(u_1, v_1, u_2, v_2, \dots, u_n, v_n, w, u_1)$ where $F_i = \{u_i, v_i\}$ for $i \in [1, n]$ and $F_0 = \{u_1, w\}$. A constraint is that the sampled vertex (attribute value) for u_1 must have the smallest degree among the sampled vertices, where the degree of a vertex a is $|R \bowtie a|$ (note that $R = R_{F_i}$ for every i). In the first call to GENERICCARDEST, I contains all u_i 's and v_i 's except w , $\Omega_I = \prod_{i \in [1, n]} R_{F_i}$, and $k = 1$; a sequence of n edges $(\{a_1, b_1\}, \dots, \{a_n, b_n\})$ is sampled where u_i and v_i map to a_i and b_i . When GENERICCARDEST is called for the second time, $I = \{w\}$. SSTE and SUST differ here.

For SSTE, $\Omega_I = \pi_I(R_{F_0} \bowtie a_1)$, $P(\cdot) = \frac{1}{|\Omega_I|}$, and k is $\lceil \frac{|\Omega_I|}{\sqrt{|R|}} \rceil$. Therefore, k vertices are uniformly sampled from Ω_I with replacement. For SUST, k is fixed to 1, and Ω_I is defined based on the degree of a_1 . If the degree is smaller than $\sqrt{|R|}$, then $\Omega_I = \pi_I(R_{F_0} \bowtie a_1)$ and $P(\cdot) = \frac{1}{\sqrt{|R|}}$. If the degree is larger than $\sqrt{|R|}$, then $\Omega_I = \pi_I R_{F_0}$ and $P(s_I) = \frac{|R \bowtie s_I|}{|R|}$. Then, at Line 7, 1) any s_I with a smaller degree than a_1 is rejected due to the constraint, and 2) an accepted s_I is further kept with probability $\frac{\sqrt{|R|}}{|R \bowtie s_I|}$, which is less than 1 from $|R \bowtie s_I| \geq |R \bowtie a_1| \geq \sqrt{|R|}$. Therefore, $P(s_I)$ is again $\frac{|R \bowtie s_I|}{|R|} \cdot \frac{\sqrt{|R|}}{|R \bowtie s_I|} = \frac{1}{\sqrt{|R|}}$. For both SSTE and SUST, if $(a_1, b_1, \dots, a_n, b_n, s_{\{w\}}, a_1)$ forms a cycle in the database, the recursion proceeds to the remaining components.

SUST guarantees the uniformity of sampling odd cycles and stars. To sample an odd cycle, SUST relies on the constraint that a_1 has the smallest degree. This is allowed in unlabeled graphs since other non-qualifying cycles (e.g., a_2 having the smallest degree) can be found by permuting the vertices and edges (e.g., constrain w.r.t. a_1 and let $a_2 = a_1$). The new proof of SSTE also uses this constraint to bound the variance (see §E in our full paper [26]). However, in non-identical relations, we cannot permute the edges with different labels.

The essence of component-at-a-time sampling is to use the *divide-and-conquer* approach by breaking a query $\mathcal{H}$ into smaller components (i.e., odd cycles and stars), solve simpler problems for these components, and estimate back the cardinality of the original query. Lemmas 8 and 9 allow us to set an induction hypothesis that bounds the variance of estimates in simpler problems. We explain these lemmas in §D.1 and extend them for labeled graphs in §D.2.

D.1 For Unlabeled Graphs

Lemma 8. (Decomposition Lemma) [6]: Given a query hypergraph $\mathcal{H}(\mathcal{V}, \mathcal{E})$ of identical binary relations (i.e., $R_F = R : \forall F \in \mathcal{E}$), the LP in Definition 1 admits a half-integral optimal solution x where 1) $x_F \in \{0, \frac{1}{2}, 1\} : \forall F \in \mathcal{E}$ and 2) the support of x , $\text{supp}(x) := \{F \in \mathcal{E} | x_F > 0\}$, forms a set of vertex-disjoint odd cycles (i.e., each cycle has an odd number of edges) and stars.

Lemma 9. [6]: Suppose that $\mathcal{H}$ (with identical binary relations as $\mathcal{E}$) is decomposed into odd cycles and stars by a half-integral optimal solution x in Lemma 8. For arbitrary subsets of odd cycles $\mathbb{C}$ and stars $\mathbb{S}$ of $\mathcal{H}$, let $\mathcal{H}_s = (\mathcal{V}_s, \mathcal{E}_s)$ be the induced subquery of $\mathcal{H}$ on vertices of $\mathbb{C}$ and $\mathbb{S}$. Then, for output size OUT_s of $\mathcal{H}_s$, $OUT_s \leq |R|^{\sum_{F \in \mathcal{E}_s} x_F}$, and $\sum_{F \in \mathcal{E}_s} x_F = \sum_{C \in \mathbb{C}, \mathcal{E}_C \in \mathbb{C}} \sum_{F \in \mathcal{E}_C} x_F + \sum_{S \in \mathbb{S}, \mathcal{E}_S \in \mathbb{S}} \sum_{F \in \mathcal{E}_S} x_F$.

D.2 For Labeled Graphs

We extend Lemma 8 to Lemma 10 for labeled graphs. Please refer to our full paper [26] for all proofs.

Proposition 10. The LP in Definition 1 admits an integral optimal solution if $\mathcal{H}$ is a bipartite graph, where relations are binary but not necessarily identical.

Therefore, we can easily remove the identity assumption on relations if $\mathcal{H}$ is bipartite. The following proposition fills the remaining non-bipartite case.

Proposition 11. The LP in Definition 1 admits a half-integral optimal solution if $\mathcal{H}$ is a non-bipartite graph, where relations are not necessarily identical.

Finally, we state Lemma 10. The extension of Lemma 9 for labeled graphs, can be directly proven by Lemma 10 as the proof by Assadi et al. [6].

Lemma 10. (Extended Decomposition Lemma) Given a query hypergraph $\mathcal{H}$ of binary relations, the LP in Definition 1 admits a half-integral optimal solution that satisfies the conditions in Lemma 8.

Assadi et al. [6] do not present a concrete join size estimator for labeled graphs but prove a lower bound $\Omega(\frac{AGM(\mathcal{H}_U)}{OUT})$. Here, $\mathcal{H}_U$ represents $\mathcal{H}$ without labels, and $AGM(\mathcal{H}_U) \geq AGM(\mathcal{H})$ since $|R_F| \leq |R|$ for each $F \in \mathcal{E}$; R_F 's are subsets of the set of all data edges R .